MIFA: Missing-Aware Filtered Approximate Nearest Neighbor Search over Vector-Scalar Hybrid Data

Weilong Ren[1], Mengyi Yan[2], Yutong Ye[3]
[1]Shenzhen Institute of Computing Sciences [2]Shandong University [3]Zhejiang Sci-Tech University
renweilong@sics.ac.cn, yanmy@sdu.edu.cn, ytye@zstu.edu.cn

***Abstract*—Filtered approximate nearest-neighbor (ANN) search retrieves the tuples nearest an embedding query subject to scalar predicates, but existing methods typically assume that all predicate attributes are observed. In real-world catalogues, predicate attributes can be *unobserved* in the indexed relation. Dropping such tuples can miss true answers, while inaccurate imputation can admit ineligible ones. We formalize *incomplete filtered ANN search* over numerical and categorical predicates and present MIFA, a framework that infers predicate eligibility without reconstructing missing values. MIFA makes hard decisions from observed values and valid deterministic constraints. For unresolved tuples, support templates combine scalar and embedding-local evidence, backing off to broader contexts when local support is sparse. A distance-ordered executor invokes statistical inference only when a tuple can still affect the top-k and uses safe geometric bounds to prune noncompetitive regions; CPU/GPU candidate streams further reduce intermediate state and data movement. We establish conditional hard-decision soundness, exact execution over the fixed predicted-eligible set, and a $(1+\epsilon)$ distance-boundary guarantee for approximate execution. Experiments on ten real-world hybrid workloads with controlled missingness validate MIFA's effectiveness and efficiency. Code artifacts and full version are available[1].**

I. Introduction

Filtered (a.k.a. *hybrid*) vector queries retrieve the tuples nearest an embedding query, subject to structured predicates. They power product search, entity-centric analytics, and retrieval-augmented applications [1], [2], [3]. Recent filtered approximate nearest-neighbor (ANN) systems interleave vector traversal with scalar filtering [4], [5], [6], [7], but they typically assume that every predicate attribute is observed, so predicate evaluation always yields a decidable `TRUE`/`FALSE`.

Real-world catalogues often violate this assumption. A record may have a searchable description and embedding, while some attributes needed for filtering are missing from its structured fields. We focus on *unobserved* attributes: the tuple has a meaningful underlying value, but that value is absent from the queried relation. Under SQL's three-valued logic, comparisons involving missing values usually evaluate to `UNKNOWN` [8], [9]. If the system drops tuples whose predicate outcome is unresolved, it can exclude records that would qualify under complete data.

Example 1: Consider retrieving the k products closest to the text query *"a small cup of curd"*, subject to `net_weight` $\leq 250\,\mathrm{g}$. In a real-world OpenCatalog snapshot [10], the product *Mother Dairy Dahi Small Can* (GTIN `8901648096986`) has no recorded quantity, although a barcode-linked external listing identifies it as a $200\,\mathrm{g}$ cup. Dropping the product due to its missing quantity removes a qualifying neighbor from consideration. The issue is not that the quantity is undefined, but that it is unobserved in the searchable catalogue. □

A natural response is to reconstruct missing values before filtering. External lookup or extraction may resolve some records exactly, and statistical imputation can estimate values from available evidence. However, filtered ANN search only requires deciding predicate *eligibility*, not recovering exact attribute values. Inaccurate reconstruction can flip the eligibility decision. Worse still, eagerly completing all missing cells wastes work on attributes not referenced by the query and on tuples that are too distant to enter the top-k.

Lazy imputation (*e.g.,* [11], [12]) reduces unnecessary reconstruction by deferring imputation inside a query plan. However, *incomplete filtered ANN search* makes this idea *rank-aware*: eligibility must be decided along a distance-ordered scan whose top-k boundary moves as nearer vectors are discovered. Simply applying relational lazy imputation is therefore insufficient. To be both correct and fast, a system must (i) estimate unresolved predicate eligibility accurately, and (ii) provably skip any uncertainty whose resolution cannot change the final top-k answers. We therefore ask: *How should a vector database answer filtered ANN queries when the scalar attributes used for filtering are incomplete?*

Challenges. Answering this question raises two challenges. First, eligibility inference must make reliable decisions from limited evidence. When observed values or hard rules (*e.g.,* schema/domain constraints) cannot determine a predicate, the system must rely on supporting records that share scalar attributes or lie nearby in the embedding space. However, narrow contexts may yield too few examples, while broad contexts can mix dissimilar records and dilute the signal. The challenge is to select informative support for each predicate and tuple, and to recognize when the available evidence is insufficient for a reliable decision.

Second, eligibility inference must be executed selectively. Evaluating every unresolved tuple can be costly, yet filtering only a fixed candidate set can miss qualifying neighbors beyond that set. Moreover, a tuple that appears competitive early may become irrelevant as closer eligible tuples are discovered. The challenge is to determine which unresolved tuples still

[1]https://github.com/authurlord/MIFA_Release

require inference and when search can safely stop, while preserving the exact top-k under a fixed eligibility decision rule.

Our proposal. We propose MIFA, a ***M**issing-aware **I**nference-guided **F**iltered **A**NN Search* framework. MIFA first resolves predicate outcomes whenever observed values or applicable deterministic rules are decisive. For unresolved tuples, it estimates predicate satisfaction using *support templates*, which select historical records based on observed scalar attributes and embedding neighborhoods. When a context contains too few supporting records, MIFA backs off to broader contexts and abstains if the evidence remains insufficient.

Moreover, MIFA integrates this inference mechanism into distance-ordered search, invoking statistical inference only when an unresolved tuple could still affect the answer. Once k tuples have been judged eligible, their farthest distance defines the current top-k boundary. As closer eligible tuples are discovered, this boundary tightens. Geometric lower bounds identify tuples and regions that cannot improve the current answer, allowing them to be pruned without resolving their eligibility. Search terminates when no unvisited tuple can improve the answer. Thus, inference determines which tuples are accepted, while exact execution returns their top-k under a fixed eligibility decision rule and avoids unnecessary eligibility evaluations and distance computations.

Contributions. We make the following contributions.

(1) Problem formulation (Section III). We formulate *incomplete filtered ANN search* over numerical and categorical predicates with unobserved attribute values. We define the complete-data top-k target and distinguish eligibility-inference error from search error.

(2) Context-adaptive eligibility inference (Sections IV-A–IV-B). We present MIFA, which infers predicate eligibility without reconstructing missing values. Its support templates estimate unresolved predicate conjunctions from observed scalar attributes and embedding-local evidence, using hierarchical backoff when local support is sparse. A shared index retrieves supporting records, while a validation-driven offline procedure selects compatible policies and structural configurations. The procedure then refits the selected policy and calibrates its acceptance threshold on separate held-out data.

(3) Inference-aware execution & guarantees (Sections IV-C–IV-E). We develop a unified executor that combines ANN seeding with safe geometric pruning, invoking statistical inference only when an unresolved tuple can affect the current top-k. We use CPU/GPU candidate streams to reduce intermediate state and data movement. We establish hard-decision soundness under valid deterministic constraints, exact execution over the fixed predicted-eligible set, and a $(1+\epsilon)$ distance-boundary guarantee for approximate execution. A base–delta design supports appended tuples, while preserving fixed base inference contexts and support evidence between refreshes.

(4) Experimental evaluation (Section V). We evaluate MIFA on ten real-world hybrid workloads with controlled missingness against missing-value handling and filtered vector-search baselines. The experiments assess recovery of the complete-data top-k, separately measure inference quality and execution efficiency, and characterize scalability, preparation costs, memory usage, and append maintenance.

Section II reviews related work, and Section VI concludes.

II. Related Work

Filtered vector search. Filtered vector search combines vector ranking with structured predicates through pre-, post-, or in-filtering [13], [1]. Representative methods include ACORN [4], NaviX [14], SIEVE [7], and FAVOR [15]. Specialized methods support categorical labels (UNG [6]) or numerical ranges (UNIFY, iRangeGraph, DSG, and WoW [5], [16], [17], [18]). A recent study evaluates hybrid search systems [19]. These methods optimize search when predicate outcomes are determined by observed scalar values. Exploiting vector–filter correlation improves search efficiency but does not itself determine predicates involving missing values. MIFA adds eligibility inference for such predicates and uses ANN indexing for candidate seeding.

Incomplete and probabilistic query processing. Incomplete database semantics distinguish certain from possible answers [20], while SQL uses three-valued logic for null-containing predicates [8], [9]. Probabilistic databases use uncertainty models to compute answer probabilities [21]. QPIAD uses mined dependencies and value distributions to rewrite queries and retrieve likely answers from incomplete autonomous databases [22]. MIFA shares the goal of recovering qualifying tuples despite missing values. It directly estimates unresolved predicate conjunctions from scalar and embedding-local evidence, then ranks accepted tuples by their observed embedding distances. Its target is the complete-data filtered top-k, rather than certain-answer enumeration or ranking by answer probability.

Top-k retrieval with costly or hidden values. Ukkonen [23] studies approximate top-k retrieval under a linear ranking function when hidden attribute values can be obtained through costly probes. Regression models assess whether partially observed rows can enter the top-k, reducing unnecessary accesses. MPro [24] likewise identifies necessary probes for ranked queries with expensive predicates. Distance browsing [25] provides incremental nearest-neighbor retrieval through a unified priority queue of spatial nodes and objects. MIFA builds on selective evaluation and best-first traversal. It addresses uncertainty in scalar-filter eligibility over observed embeddings, without retrieving the missing values. Context-adaptive inference defines eligibility, while geometric search preserves the top-k under the fixed decision rule.

Uncertain nearest-neighbor search. Probabilistic nearest-neighbor methods rank objects by their probability of being nearest under uncertain existence or location [26]. MIFA assumes observed embeddings and treats uncertainty in scalar-filter eligibility. Accepted tuples are ranked by

embedding distance, and execution guarantees are relative to the fixed predicted-eligible set.

Missing-value estimation and lazy imputation. General-purpose methods reconstruct missing cells using chained equations [27] or mixed-type nonparametric imputation [28]. Data-cleaning systems support probabilistic repair (HoloClean [29]), error detection (Raha [30]), and context-based correction (Baran [31]). These methods target cell reconstruction or cleaning rather than filtered vector retrieval. ImputeDB places imputation operations within relational query plans to balance quality and cost [11]. ZIP uses imputation-aware operators to defer reconstruction while preserving the results of eager imputation [12]. MIFA shares their goal of avoiding unnecessary missing-data processing, but directly estimates predicate eligibility without materializing repaired values. It combines support-based backoff and abstention with distance-ordered search, invoking statistical inference only for tuples that remain top-k-competitive.

III. Problem Definition

This section introduces hybrid data and filtered ANN queries, defines their complete-data target when scalar values are missing, and formalizes incomplete filtered ANN search.

Hybrid data and filtered ANN queries. We consider a relation D with an embedding attribute $A_{\mathbf{e}} \in \mathbb{R}^d$ and numerical or categorical scalar attributes $\bar{\mathcal{A}} = \{A_1, \ldots, A_m\}$. Embeddings are always observed, whereas scalar values may be missing, denoted by $t[A_i] = \perp$. Following prior work (*e.g.,* [4]), we assume a single embedding attribute for clarity; multiple embedding attributes can be handled analogously.

A filtered ANN query is $q = (\mathbf{q}_e, k, \phi)$, where $\mathbf{q}_e$ is the query embedding, k is the requested result size, and $\phi = \bigwedge_{j=1}^{\ell} \psi_j$ is a predicate over scalar attributes. Each atom ψ_j is a numerical comparison or range, or a categorical equality or set-membership condition. Set membership captures categorical disjunctions; arbitrary Boolean formulas are outside our scope. Tuples are ordered by increasing distance to $\mathbf{q}_e$:

$$t_i \preceq_q t_j \iff \mathsf{dist}(\mathbf{q}_e, t_i[A_{\mathbf{e}}]) \le \mathsf{dist}(\mathbf{q}_e, t_j[A_{\mathbf{e}}]).$$

When all scalar attributes are observed, the exact answer is

$$\mathsf{Ans}(q, D) = \mathrm{TopK}_k\big(\{t \in D \mid \phi(t[\bar{\mathcal{A}}]) = \mathsf{true}\}, \preceq_q\big),$$

where $\mathrm{TopK}_k(S, \preceq_q)$ returns the k closest tuples in S under $\preceq_q$, or all tuples if $|S| < k$.

Incomplete-data semantics and target. With missing values, predicates follow three-valued semantics: $\phi(t[\bar{\mathcal{A}}]) \in \{\mathsf{true}, \mathsf{false}, \mathsf{unknown}\}$. It evaluates to false if any atom is false, to true if all atoms are true, and to unknown otherwise.

We assume that each missing entry has a well-defined underlying value that is unobserved in D; undefined or inapplicable values are outside our scope. Let D^* denote the unobserved complete relation underlying D. It has the same tuples and embeddings, agrees with D on all observed scalar values, and assigns each missing entry its underlying value. For each $t \in D$, let t^* denote its complete counterpart in D^*. The complete-data target is

$$E^*(q, D^*) = \{t \in D \mid \phi(t^*[\bar{\mathcal{A}}]) = \mathsf{true}\},$$
$$\mathsf{Ans}^*(q, D^*) = \mathrm{TopK}_k\big(E^*(q, D^*), \preceq_q\big).$$

The complete-data answer $\mathsf{Ans}^*(q, D^*)$ serves as the reference for evaluating answers computed from D (see below).

Incomplete filtered ANN search. Given an incomplete relation D and a filtered ANN query $q = (\mathbf{q}_e, k, \phi)$, *incomplete filtered ANN search* returns at most k tuples intended to match the complete-data target, *i.e.,* tuples whose complete counterparts satisfy ϕ and rank closest to $\mathbf{q}_e$ under $\preceq_q$. We denote the returned answer by

$$\widehat{\mathsf{Ans}}(q, D) \subseteq D, \qquad |\widehat{\mathsf{Ans}}(q, D)| \le k.$$

For queries with nonempty targets, we define $\mathrm{TP} = \big|\widehat{\mathsf{Ans}}(q, D) \cap \mathsf{Ans}^*(q, D^*)\big|$, $\mathrm{Precision} = \mathrm{TP}/|\widehat{\mathsf{Ans}}(q, D)|$, $\mathrm{Recall} = \mathrm{TP}/|\mathsf{Ans}^*(q, D^*)|$, and $F_1 = 2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}/(\mathrm{Precision} + \mathrm{Recall})$. We set Precision to 0 if $\widehat{\mathsf{Ans}}(q, D) = \emptyset$, and $F_1 = 0$ if both precision and recall are 0.

Our objective is to achieve high answer F_1 relative to $\mathsf{Ans}^*(q, D^*)$ while limiting eligibility-inference and vector-search costs. For a fixed eligibility decision rule, exact execution returns the top-k among tuples judged eligible. Section IV-C formalizes exact and approximate execution over this predicted-eligible set.

IV. The MIFA Framework

MIFA couples a fixed predicate-eligibility policy with inference-aware, distance-ordered vector search. Section IV-A overviews its architecture; Sections IV-B and IV-C present offline policy/index construction and inference-aware execution, respectively; Section IV-D describes CPU/GPU acceleration; and Section IV-E establishes correctness and approximation guarantees and complexity bounds.

A. Overview

Design overview. MIFA is built on two design principles. (1) Filtered queries require determining predicate satisfaction rather than reconstructing missing attribute values, so MIFA infers eligibility directly from available evidence. (2) Statistical inference is invoked only for unresolved tuples that could influence the answer. Evaluating every unresolved tuple wastes work, while filtering a fixed ANN candidate set can miss qualifying neighbors. MIFA therefore couples on-demand inference with distance-ordered search and safe geometric pruning.

Framework. Figure 1 illustrates the MIFA framework, while Figure 2 summarizes its construction and execution. Given an incomplete relation D and a filtered kNN query $q = (\mathbf{q}_e, k, \phi)$, MIFA returns at most k tuples closest to $\mathbf{q}_e$ that are predicted to satisfy ϕ. In the offline phase, MIFA learns a reusable eligibility policy and builds supporting indexes; in the online phase, it applies them selectively during query processing.

(1) Offline construction (Section IV-B). MIFA constructs can-

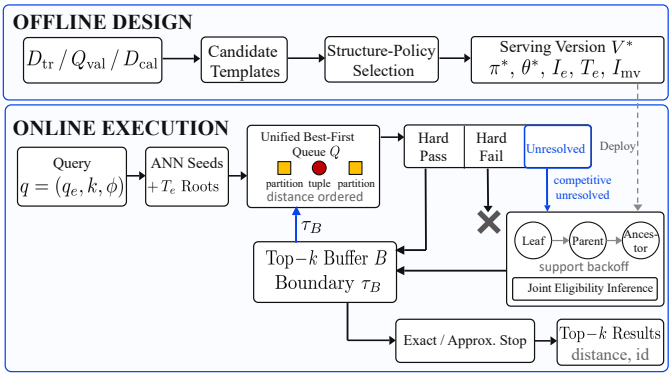


Fig. 1: Architecture of MIFA.

didate support templates from the training relation D_{tr}. Each template specifies how to retrieve supporting records from D_{tr} using observed scalar attributes and embedding-local contexts. Using a validation split and previous workloads, MIFA jointly selects a structural configuration $\Gamma^\star$ (which determines the context hierarchy) and a policy design $\pi^\star$ (which specifies template selection and eligibility scoring) (Lines 1–2). MIFA then builds (a) a partition tree $\mathcal{T}_e$ with MBR distance lower bounds for safe pruning, (b) a shared support index $\mathcal{I}_{\text{mv}}$ for template-based evidence retrieval, and (c) a plug-in ANN index $\mathcal{I}_e$ for seeding nearby candidates. We denote these compatible serving-time structures by $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$. MIFA refits the selected policy $\pi^\star$ for deployment and calibrates its decision threshold $\theta^\star$ on separate calibration data. Finally, it publishes $(\mathcal{J}, \pi^\star, \theta^\star)$ as a single serving version (Lines 3–7).

(2) Online execution (Section IV-C). Given a query q, MIFA pins a read-consistent serving version (Line 8). For a candidate tuple, MIFA first checks whether observed values and valid deterministic constraints (*e.g.,* a ZIP code uniquely determines a city) imply a sound hard pass or fail. Otherwise, MIFA applies $\pi^\star$: it selects a template, retrieves supporting records, and estimates whether the unresolved predicates hold jointly. For the selected template, insufficient support triggers backoff to broader ancestor contexts. With sufficient support, MIFA compares the mapped eligibility score to $\theta^\star$; otherwise, it abstains and treats the tuple as ineligible.

MIFA applies this decision rule within a distance-ordered best-first search. It uses the ANN index to supply nearby seeds, and hard-pass seeds initialize the answer buffer B (Line 9). A partition tree covers all searchable tuples and stores projected minimum bounding rectangles (MBRs) that lower-bound distances to descendants. Embedding-local contexts select evidence for inference, while MBRs provide safe geometric bounds. MIFA maintains a unified priority queue over tree nodes and unresolved tuples (Line 10). Once B contains k predicted-eligible tuples, the farthest distance among them defines the current boundary. MIFA prunes entries whose distance bounds exceed this boundary and performs statistical inference only when an unresolved tuple reaches the queue head and remains competitive. In exact mode ($\epsilon = 0$), MIFA continues the best-first traversal until the queue is exhausted

Offline input: An incomplete relation D; disjoint training, validation and calibration relations D_{tr}, D_{val}, and D_{cal}; and workloads $\mathcal{Q}_{\text{val}}$ and $\mathcal{Q}_{\text{cal}}$.
Online input: A query $q = (\mathbf{q}_e, k, \phi)$ and parameter $\epsilon \geq 0$.
Output: $\widehat{\mathsf{Ans}}(q, D)$.
/* Offline construction */
1. $\mathcal{T}_{\text{cand}} := \mathsf{ConstructTemplates}(D_{\text{tr}}, \mathcal{Q}_{\text{val}})$;
2. $(\Gamma^\star, \pi^\star) := \mathsf{SelectJointDesign}(\mathcal{T}_{\text{cand}}, D_{\text{tr}}, D_{\text{val}}, \mathcal{Q}_{\text{val}})$;
3. $(\mathcal{I}_e, \mathcal{T}_e) := \mathsf{BuildSearchStructures}(D, A_{\mathbf{e}}; \Gamma^\star)$;
4. $\mathcal{I}_{\text{mv}} := \mathsf{BuildSupportIndex}(D_{\text{tr}}, \mathcal{T}_e, \pi^\star)$;
5. $\pi^\star := \mathsf{RefitDeploymentPolicy}(\pi^\star, D_{\text{tr}}, D_{\text{val}}, \mathcal{Q}_{\text{val}}, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$;
6. $\theta^\star := \mathsf{CalibrateThreshold}(\pi^\star, D_{\text{cal}}, \mathcal{Q}_{\text{cal}}, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$;
7. $\mathsf{PublishVersion}(D, \mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\text{mv}}, \pi^\star, \theta^\star)$;
/* Online execution */
8. $V_q := \mathsf{PinReadVersion}()$;
9. $(B, S_{\text{reg}}) := \mathsf{HardDecisions}(\mathsf{RetrieveCandidates}(q, V_q), q, V_q)$;
10. $B := \mathsf{UnifiedTraverseAndInfer}(q, \epsilon, B, S_{\text{reg}}, V_q)$;
11. **return** B under $\preceq_q$;

Fig. 2: The workflow of MIFA.

or all remaining entries are pruned by the current boundary, yielding the exact top-k over the predicted-eligible set defined by $(\pi^\star, \theta^\star)$. In approximate mode ($\epsilon > 0$), it may terminate early once certifying that no unseen predicted-eligible tuple can beat the boundary by more than a factor of $(1 + \epsilon)$.

Updates. Newly appended tuples are searched via a delta tree using fixed base inference contexts and support evidence until the next refresh (Section IV-C).

Example 2: Continuing Example 1, consider a filtered 2NN query for the products closest to *"a small cup of curd"* under $\mathsf{net_weight} \leq 250\,\text{g}$. The distances and inference scores below are illustrative. Let $\theta^\star = 0.75$ and $\epsilon = 0$. Suppose the ANN probe returns t_1 and t_2 at distances 0.18 and 0.24, and both satisfy the predicate by observed weight, yielding hard-pass decisions. They initialize the answer buffer $B = \{t_1, t_2\}$ with boundary $\tau_B = 0.24$. During traversal, *Mother Dairy Dahi Small Can* (t_3) reaches the queue head at distance 0.14, but its missing weight leaves eligibility unresolved. Since t_3 is competitive, MIFA retrieves supporting records and applies $\pi^\star$. Suppose it infers $\widehat{p}(t_3) = 0.82 > \theta^\star$. MIFA admits t_3, replaces t_2, and tightens the boundary to $\tau_B = 0.18$.

If all remaining queue entries have distance lower bounds above 0.18 (*e.g.,* the queue head is bounded by 0.21), then no unseen tuple can improve B. MIFA terminates and returns $\{t_3, t_1\}$ without evaluating eligibility for non-competitive tuples or reconstructing missing values. □

Remark. MIFA breaks ties between equal-distance tuples using a fixed unique tuple identifier. This ordering is used consistently for all target answers and query execution.

B. Offline Design and Index Construction

Inputs and outputs. The offline phase takes as input (a) the incomplete relation D; (b) mutually disjoint development relations D_{tr}, D_{val}, and D_{cal} with disjoint tuple/entity identities; and labeled workloads $\mathcal{Q}_{\text{val}}$ and $\mathcal{Q}_{\text{cal}}$. Specifically, MIFA uses

D_{tr} for template discovery, statistical support, and partition-topology construction; D_{val} for design selection and score-map fitting; and D_{cal} for threshold selection after the policy and structures are fixed. It builds searchable indexes over D and the support index over D_{tr}. The output is a read-consistent serving version comprising D, compatible structures $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\mathsf{mv}})$, a deployment policy $\pi^\star$, and a threshold $\theta^\star$.

Evidence and Candidate Templates. Given a tuple–predicate pair (t, ϕ), we first apply observed values and valid deterministic constraints (*e.g.,* conditional functional dependencies [32]) to resolve as much of ϕ as possible; for the remaining unresolved atoms, we define reusable support templates, explain how they are mined from D_{tr}, and form a finite candidate pool.

Support templates. Let $\phi_{\mathsf{res}}(t)$ be the conjunction of atoms in ϕ that remain unresolved for tuple t, and let $Y_\phi(t)$ be the missing attributes referenced by $\phi_{\mathsf{res}}(t)$. A template $r : X_s, [\mathsf{Loc}(A_{\mathbf{e}})] \rightsquigarrow Y$ specifies an observed scalar body X_s, target attributes Y, and scalar matching conditions, with $X_s \cap Y = \emptyset$. It applies when its body conditions are evaluable on t and $Y_\phi(t) \subseteq Y$. Supporting records must match the body and have $X_s \cup Y$ observed. The optional $\mathsf{Loc}(A_{\mathbf{e}})$ restricts support to an embedding-local context; omitting it gives a scalar-only template. For Example 1, a template may match brand and category and use observed net_weight values to estimate satisfaction of net_weight $\leq 250\,\mathrm{g}$.

MIFA applies dependency discovery to D_{tr} (*e.g.,* [32] and [33]) to surface statistically informative candidate templates, specifying scalar body attributes and matching conditions; it then augments each retained scalar template with a location-aware variant that restricts support to an embedding-local context via $\mathsf{Loc}(A_{\mathbf{e}})$. After filtering templates with insufficient support, MIFA forms the finite pool $\mathcal{T}_{\mathsf{cand}}$ by including scalar-only templates, location-aware templates, and body-free fallbacks ($X_s = \emptyset$). See Appendix A in [34] for details.

Inference and Search Structures. MIFA uses a partition tree $\mathcal{T}_e$ for inference contexts and safe pruning, a shared support index $\mathcal{I}_{\mathsf{mv}}$ for template-based evidence retrieval, and a plug-in ANN index $\mathcal{I}_e$ for candidate seeding.

Projected partition tree. MIFA learns an orthogonal PCA projection and partition topology from D_{tr}. The tree defines embedding-local contexts for location-aware templates and provides geometric bounds for search. The configuration $\Gamma = (d_p, B_{\mathsf{leaf}})$ specifies the projection dimension and training leaf capacity. Training and target tuples follow the same split tests: a target's leaf-to-root path defines nested contexts, and the training tuples within each context form its support population.

Under the selected configuration, MIFA routes D through the fixed topology to populate $\mathcal{T}_e$. Leaves store searchable tuples, and each node N stores a projected minimum bounding rectangle $\mathsf{MBR}(N)$ enclosing its searchable descendants. Thus, D_{tr} determines the contexts through the split tests, while D determines searchable contents and MBRs. Since orthogonal projection cannot increase Euclidean distances, the distance from the query's projection to $\mathsf{MBR}(N)$ lower-bounds the original query-to-tuple distance for every descendant t:

$$\mathsf{LB}(\mathbf{q}_e, N) \leq \mathsf{dist}(\mathbf{q}_e, t[A_{\mathbf{e}}]).$$

For cosine distance, we use the analogous bound for ℓ_2-normalized embeddings. Once the answer buffer has k predicted-eligible tuples, MIFA prunes nodes whose metric-specific bounds exceed the buffer's current farthest distance. See Appendix A in [34] for tree construction and bounds.

Shared support index. The index $\mathcal{I}_{\mathsf{mv}}$ retrieves training records from D_{tr} with observed body and target attributes that satisfy the selected template's scalar matching conditions and, when required, lie in the chosen embedding-local context. Scalar-only templates use the global matching population. Retrieved target values are tested against the current residual predicates, allowing evidence reuse across query constants. See Appendix B in [34] for index construction and retrieval details.

Plug-in ANN index. MIFA builds $\mathcal{I}_e$ over D using an existing ANN method (*e.g.,* HNSW [35]). Each seed undergoes hard eligibility checks using observed values and valid constraints. Hard-pass seeds update the answer buffer; hard-fail seeds are discarded, and unresolved seeds are deferred to tree traversal. The tree provides complete coverage, so ANN seeding affects efficiency but not correctness.

Structure–Policy Selection and Finalization. Since partitioning affects inference contexts and search cost, MIFA jointly selects a structural configuration Γ and policy design π on D_{val}, and then selects the deployment threshold $\theta^\star$ on D_{cal}.

Eligibility policy. A policy π retains ordered templates from $\mathcal{T}_{\mathsf{cand}}$ and specifies support requirements, context-backoff sequences, and score-map fitting rules. It selects the first applicable template, prioritizing richer observed scalar bodies and using body-free fallbacks. A location-aware template r uses the most local context along its fixed leaf-to-ancestor path meeting its minimum support size $s_{\min}(r)$; scalar-only templates retrieve matching support records from D_{tr}, without an embedding-local restriction. If no permitted context provides sufficient support, π abstains.

For the selected template r, MIFA computes the fraction of matching training records that satisfy all unresolved predicates in ϕ and uses it to derive the raw eligibility estimate $z_r(t, \phi)$ for tuple t. A fitted map $g_r : [0,1] \to [0,1]$ converts this estimate into the final eligibility score under policy π: $\widehat{p}_\pi(t, \phi) := g_r(z_r(t, \phi))$. Each map is fitted from labeled, non-abstaining validation instances assigned to its template. Cross-fitting evaluates each instance using a map fitted without its fold. Under the deployed policy $\pi^\star$, a non-abstaining tuple is predicted eligible iff $\widehat{p}_{\pi^\star}(t, \phi) \geq \theta^\star$, where $\theta^\star$ is the calibrated acceptance threshold; abstentions are treated as ineligible.

Joint selection. Let $\mathcal{C}$ denote the finite set of candidate pairs (Γ, π) induced by $\mathcal{T}_{\mathsf{cand}}$, candidate structural configurations, and predeclared policy choices that satisfy coverage requirements across predicate–missingness cases. For $(\Gamma, \pi) \in \mathcal{C}$, $\mathsf{InfScore}(\Gamma, \pi)$ measures cross-fitted eligibility-estimation quality on held-out validation folds (higher is better), while

$\mathsf{TimeCost}(\Gamma, \pi)$ measures normalized executor replay cost on validation queries under predeclared representative thresholds (lower is better). See Appendix A in [34] for more details. MIFA then selects

$$(\Gamma^\star, \pi^\star) = \arg\max_{(\Gamma,\pi)\in\mathcal{C}} \big[\mathsf{InfScore}(\Gamma, \pi) - \lambda\, \mathsf{TimeCost}(\Gamma, \pi)\big],$$

where $\lambda \geq 0$ controls the quality–cost trade-off.

Finalization and calibration. With $\Gamma^\star$ and the policy design fixed, MIFA builds the persistent structures, recomputes raw validation estimates under the final hierarchy, and refits each map using all non-abstaining validation instances assigned to its template. These maps complete the deployment policy $\pi^\star$. With the policy and structures fixed, MIFA selects $\theta^\star$ from a predeclared finite set Θ by minimizing mean calibration-group loss on D_{cal} (*e.g.,* $1 - F_1@k$), and then publishes the serving version. See Appendix A in [34] for more details.

C. Inference-Aware Filtered ANN Execution

Inputs and outputs. Given a query $q = (\mathbf{q}_e, k, \phi)$, an approximation parameter $\epsilon \geq 0$, and the fixed serving version $(\mathcal{J}, \pi^\star, \theta^\star)$ for D from Section IV-B, MIFA returns a buffer B of at most k predicted-eligible tuples ordered by $\preceq_q$.

High-level idea. Figure 3 summarizes the executor. MIFA combines ANN seeding with best-first tree traversal and invokes statistical inference only for unresolved tuples that can still affect B. It uses geometric bounds to prune noncompetitive regions and determine when search can safely stop.

On-Demand Policy Application. MIFA applies hard checks using observed values and deterministic constraints. For unresolved tuples, it defers statistical inference until they reach the queue head and remain competitive. It then applies the fixed policy $\pi^\star$ and threshold $\theta^\star$ using the serving version's contexts and support records.

Hard checks and residual predicate (lines 3 and 18). Given a tuple t and predicate ϕ, MIFA evaluates atoms whose referenced values are observed in t. If any observed atom is false, it returns a hard fail; if all atoms are observed and true, it returns a hard pass. For the remaining atoms, MIFA applies Σ_{det}, the fixed set of valid deterministic constraints supplied independently of the statistical templates. Each applicable constraint restricts a missing attribute to a certified domain containing its true value. MIFA intersects these domains separately for each missing attribute. For a nonempty intersection, it certifies an atom as true if every remaining value satisfies it, or as false if none does. If no constraint applies, the intersection is empty, or both satisfying and violating values remain, the atom stays unresolved. Any certified-false atom yields a hard fail.

Let $\phi_{\mathsf{res}}(t)$ denote the conjunction of atoms in ϕ that remain unresolved after the hard checks. If none remain, MIFA returns a hard pass; otherwise, it defers statistical inference.

Statistical evaluation (lines 24–25). MIFA may first retrieve candidate values for missing predicate attributes from linked external records [36]. It uses high-confidence candidates to complete these attributes for predicate evaluation and updates the residual predicate $\phi_{\mathsf{res}}(t)$ accordingly. If eligibility remains unresolved, let $Y_\phi(t)$ denote the missing attributes referenced by $\phi_{\mathsf{res}}(t)$. Following $\pi^\star$'s fixed ordering, MIFA selects the first retained template r whose target covers $Y_\phi(t)$ and whose scalar body X_s has all values available for t, using body-free templates as fallbacks. It abstains if no template applies or the selected one has insufficient support.

For the selected template, MIFA retrieves matching training records through $\mathcal{I}_{\mathsf{mv}}$. Location-aware templates restrict support to the most local permitted context on t's leaf-to-ancestor path containing at least $s_{\min}(r)$ matches; scalar-only templates use matches from D_{tr} without an embedding-local restriction. With sufficient support, MIFA derives the raw estimate $z_r(t, \phi)$ from satisfaction of the entire residual conjunction among matching records and applies the deployed map g_r to obtain $\widehat{p}_{\pi^\star}(t, \phi) = g_r(z_r(t, \phi))$. It predicts t eligible iff this score is at least $\theta^\star$; lower scores and abstentions are treated as ineligible.

Unified Best-First Execution. ANN seeding finalizes only hard decisions, while tree traversal provides complete coverage. Nodes and unresolved tuples share a priority queue, allowing MIFA to defer statistical inference until a tuple reaches the queue head and remains competitive.

State and coverage invariant. Each tuple in D belongs to exactly one leaf of $\mathcal{T}_e$. MIFA maintains (a) a buffer B containing

Input: A query $q = (\mathbf{q}_e, k, \phi)$, a relation D, the deployed $(\pi^\star, \theta^\star)$, its compatible structures $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\mathsf{mv}})$, and $\epsilon \geq 0$.
Output: A buffer B of at most k predicted-eligible tuples in D.

```
   /* Initialize the execution state */
1.   B := ∅; S_reg := ∅;
   /* ANN seeding: finalize only hard decisions */
2.   for each t ∈ RetrieveCandidates(I_e, q_e) do
3.     h := HardEvaluate(t, φ, π*);
4.     if h = pass then B := UpdateBuffer(B, t, q);
5.     if h ∈ {pass, fail} then S_reg := S_reg ∪ {t};
   /* Unified best-first traversal */
6.   Q := InitNodeQueue(T_e, q_e);
7.   while Q ≠ ∅ do
8.     if CanStop(Q, B, k, ε) then break;
9.     x := PopMin(Q);
10.    if x is an internal node then
11.      for each child N' of x do
12.        if |B| < k or LB(q_e, N') ≤ τ_B then
13.          Push(Q, N', LB(q_e, N'));
14.    elseif x is a leaf then
15.      for each t ∈ Tuples(x) \ S_reg do
16.        d_t := dist(q_e, t[A_e]);
17.        if CanAffect(t, B, q) then
18.          h := HardEvaluate(t, φ, π*);
19.          if h = pass then B := UpdateBuffer(B, t, q);
20.          elseif h = unresolved then Push(Q, t, d_t);
21.        S_reg := S_reg ∪ {t};
22.    else  /* x is an unresolved tuple */
23.      if CanAffect(x, B, q) then
24.        ê(x, φ) := InferEligibility(x, φ, I_mv, π*, θ*);
25.        if ê(x, φ) = 1 then B := UpdateBuffer(B, x, q);
26.  return B ordered by ⪯_q;
```

Fig. 3: Inference-aware filtered ANN execution in MIFA.

up to k nearest accepted tuples found so far, (b) a priority queue Q containing tree nodes and unresolved tuples, and (c) a registration set S_{reg} recording tuples already handled or queued for statistical inference. At each iteration boundary, every tuple not finalized or safely excluded is represented by either a queued tuple or an unexpanded node in Q.

When $|B| = k$, let t_B be its lowest-ranked tuple under $\preceq_q$. Define the distance boundary and minimum queue key as

$$\tau_B = \begin{cases} \mathsf{dist}(\mathbf{q}_e, t_B[A_{\mathbf{e}}]), & |B| = k, \\ \infty, & |B| < k, \end{cases}$$

$$\lambda_Q = \begin{cases} \min_{x \in Q} \mathsf{key}(x), & Q \neq \emptyset, \\ \infty, & Q = \emptyset. \end{cases}$$

A node's key is its safe geometric lower bound; an unresolved tuple's key is its exact distance to $\mathbf{q}_e$. Thus, λ_Q lower-bounds the distance of every unprocessed tuple represented by Q. Buffer updates retain the nearest k accepted tuples under $\preceq_q$, or all accepted tuples if fewer have been found.

ANN seeding and geometric traversal (lines 2–21). MIFA retrieves seeds from $\mathcal{I}_e$ and applies hard evaluation. Hard pass updates B; both hard pass and hard fail outcomes are finalized and registered. Unresolved seeds remain unregistered and are handled by tree traversal. Since the tree provides complete coverage, exact-mode results are independent of ANN seed recall.

MIFA initializes Q with the tree root, keyed by its lower bound. When an internal node is popped, each child N' is queued unless B is full and $\mathsf{LB}(\mathbf{q}_e, N') > \tau_B$. When a leaf is popped, MIFA computes the exact distance of each unregistered tuple. A tuple remains competitive if B is not full or it ranks ahead of t_B under $\preceq_q$. Otherwise, it cannot enter the current or any subsequently improved buffer, so its eligibility need not be evaluated.

For a competitive tuple, MIFA updates B on a hard pass, discards the tuple on a hard fail, and queues it at its exact distance if eligibility remains unresolved. Every tuple examined in the leaf is registered, including noncompetitive tuples, preventing duplicate processing.

Deferred statistical inference (lines 22–25). When an unresolved tuple t reaches the queue head, MIFA rechecks whether it remains competitive, since the boundary may have tightened after t was queued. If not, MIFA discards t without inference. Otherwise, it applies the statistical stage above to obtain $\widehat{e}(t, \phi) \in \{0, 1\}$, where 1 denotes acceptance. Only an accepted tuple updates B. Thus, inference determines admission, while geometric bounds and buffer competition determine which eligibility evaluations can be avoided.

Exact and approximate modes (lines 7–8). The parameter ϵ controls the termination condition; eligibility decisions, exact-distance ranking, and safe node pruning remain unchanged.

(1) Exact mode ($\epsilon = 0$). MIFA terminates when Q is empty or when B is full and $\lambda_Q > \tau_B$. The strict inequality preserves the established ordering of distance ties. Let $\widehat{E}(q, D)$ denote the tuples accepted by the fixed eligibility rule. The returned buffer is therefore $\mathrm{TopK}_k(\widehat{E}(q, D), \preceq_q)$.

(2) Approximate mode ($\epsilon > 0$). Once B is full, MIFA may terminate early when $\tau_B \leq (1 + \epsilon)\lambda_Q$. The returned k-th distance is at most $(1+\epsilon)$ times the exact-mode k-th distance, although the tuple set may differ.

Base–delta maintenance. MIFA extends the executor in Figure 3 to append-heavy workloads using a separate delta tree, while retaining the same eligibility, pruning and stopping rules.

Base and delta state. After each refresh, let D be the published base relation and ΔD the newly appended tuples. The active relation is $D \cup \Delta D$, with disjoint tuple sets indexed by $\mathcal{T}_e$ and $\Delta\mathcal{T}_e$, respectively. The delta tree reuses the frozen projection and split tests of $\mathcal{T}_e$ but maintains its own tuples and MBRs, preserving safe bounds comparable to those of the base tree. For each append, MIFA inserts the tuple into ΔD and the ANN index (or an equivalent delta layer), routes it down a single path in $\Delta\mathcal{T}_e$, and updates counts and MBRs along that path. The base tree remains unchanged between refreshes.

Unified search and refresh. MIFA searches the base and delta trees using a shared buffer B, queue Q, and registration set S_{reg}. It retrieves ANN seeds from both relations (lines 2–5) and initializes Q with the root of each nonempty tree (line 6). All node and tuple entries then compete in the shared queue under the same traversal and stopping rules (lines 7–26). Exact mode returns the top-k over $\widehat{E}(q, D \cup \Delta D)$; approximate mode retains the $(1 + \epsilon)$ distance-boundary guarantee.

During each serving period, appended tuples use their corresponding frozen base-tree contexts and the current support index $\mathcal{I}_{\mathsf{mv}}$, built from D_{tr}. They are immediately searchable once published but contribute no statistical support within the current serving version. An append becomes visible only after its relation, ANN and delta-tree updates are published together; each query uses one consistent version throughout execution.

A periodic refresh merges $D \cup \Delta D$ into a new base and reruns the offline procedure in Section IV-B, including structure–policy selection, structure construction, policy finalization, and threshold calibration. MIFA resets ΔD and atomically publishes the new version, while in-flight queries continue using their previous versions.

D. Execution Acceleration

Although the unified executor avoids unnecessary inference and search, an unfused implementation may still materialize relation-sized distance/outcome/state arrays and shuttle them across operators or the host–device boundary, dominating latency even when Figure 3 examines only a small frontier.

Basic idea. MIFA addresses this bottleneck with *residency-preserving candidate streams*: it keeps vectors and scalar columns where they reside and moves only compact candidate records across operators or devices. It performs hard eligibility checks and distance screening locally, discarding hard failures and forwarding tuples that may improve the answer buffer B. Hard passes require no statistical inference; unresolved tuples undergo inference only when they reach the unified queue

head and remain competitive. Each stream supplies candidates incrementally and remains available for further requests. The executor merges stream frontiers and tracks distance lower bounds for tuples not yet emitted, allowing it to request more candidates as needed and stop under the condition in Section IV-C. This avoids constructing relation-sized distance and eligibility arrays for each query: intermediate candidate records are materialized as search proceeds, while resident data, indexes, and reusable postings persist across queries.

CPU execution. Under a fixed serving version, the eligibility outcome for (t, ϕ) depends on the tuple and predicate, but not on the query embedding $\mathbf{q}_e$ or k. MIFA therefore caches resolved outcomes as version-specific postings of tuple row IDs referencing resident data. These outcomes can be reused across queries sharing ϕ, but not across serving versions.

Workers stream IDs from eligible postings, compute distances directly from resident vectors, and maintain local frontiers under $\preceq_q$. Tuples without cached outcomes remain covered by unified traversal; cached ineligible outcomes require no further inference. The executor merges the frontiers, forwards only competitive candidates, and requests additional candidates as needed. This avoids per-query relation-sized distance/state arrays and corpus copies, limiting traffic to resident-data reads and compact candidate state.

GPU execution. On GPUs, MIFA keeps vectors and predicate columns device-resident and fuses distance computation, hard evaluation, and local candidate selection. It suppresses hard failures and emits candidate pages $(\mathsf{id}(t), d_t, h_t)$ under $\preceq_q$, where h_t is the hard outcome, after screening against the current answer boundary. The host merges pages with the traversal frontier and rechecks competitiveness before invoking indexed eligibility inference for an unresolved queue-head tuple.

Let M_q be the number of candidate records emitted to the host. MIFA transfers $O(M_q)$ candidate state (not a $\Theta(|D|)$ distance/state array) and keeps a compact host frontier; this bounds transferred state, not distance evaluations.

Multi-GPU execution. MIFA supports replication for throughput and partitioning for capacity. In replication mode, it places the full resident corpus on each GPU and assigns each query to one GPU, requiring no cross-device merge. In partitioned mode, it searches corpus shards in parallel and stably merges their candidate streams under $\preceq_q$. A shared answer buffer and lower bounds covering the unconsumed portion of every shard govern candidate requests and termination.

Both modes use the same serving version's policy, score maps, threshold, inference contexts, and statistical support. Corpus partitioning therefore preserves each tuple's eligibility decision. The partitioned mode exchanges compact candidate pages rather than centrally scheduling resident tuples.

E. Guarantees and Complexity

This section presents MIFA's guarantees for hard decisions and execution, and summarizes its computational and storage costs. Detailed proofs and complexity analyses are in [34].

Hard-decision soundness. If the applied constraints in Σ_{det} are sound and hard certification uses only nonempty domains, every hard pass tuple satisfies ϕ in D^*, while every hard fail tuple violates it. Under these assumptions, eligibility errors can arise only from statistical decisions or abstention.

Execution guarantees. Fix a read-consistent relation snapshot D and its compatible serving version $(\mathcal{J}, \pi^\star, \theta^\star)$, including the inference contexts and support records. For query q, let $\widehat{E}_{\pi^\star,\theta^\star}(q, D)$ denote the tuples accepted by the fixed eligibility rule, and let $\mathsf{Ans}_{\pi^\star,\theta^\star}(q, D)$ denote their exact top-k under $\preceq_q$. Let $\widehat{\mathsf{Ans}}_{\pi^\star,\theta^\star,\epsilon}(q, D)$ denote the output of Section IV-C.

Theorem 1: *With complete tree coverage and valid distance lower bounds,* MIFA *satisfies: (a) in exact mode,* $\widehat{\mathsf{Ans}}_{\pi^\star,\theta^\star,0}(q, D) = \mathsf{Ans}_{\pi^\star,\theta^\star}(q, D)$*; and (b) in approximate mode ($\epsilon > 0$), if* $|\widehat{E}_{\pi^\star,\theta^\star}(q, D)| \geq k$*, it returns k tuples with* $\tau_B \leq (1+\epsilon) d_k^{\mathsf{pred}}$*, where* d_k^{pred} *is the distance of the kth-nearest tuple in* $\widehat{E}_{\pi^\star,\theta^\star}(q, D)$*.* □

Exact execution returns the top-k induced by the fixed eligibility rule, while approximate execution bounds the returned distance boundary. Both are *w.r.t.* the predicted-eligible set.

Complexity. We summarize online query time, persistent and query-local space, and per-tuple append cost.

Online query time. Let m_v, n_v, u_q, u_v, b_v denote the numbers of node-bound computations, tuple examinations, queued unresolved tuples, statistical inferences, and buffer updates, respectively; let H denote the maximum queue size. With ANN retrieval cost C_{seed} and unit costs $c_{\mathsf{lb}}, c_v, c_{\mathsf{det}}, c_{\mathsf{inf}}$ for bounds, distances, hard checks, and inference, a conservative query-time bound is $O(dd_p + C_{\mathsf{seed}} + m_v c_{\mathsf{lb}} + (m_v + u_q)\log(H + 1) + n_v(1 + c_v + c_{\mathsf{det}}) + u_v c_{\mathsf{inf}} + b_v \log(k + 1))$.

Space complexity. Let S_{data} and S_{cache} denote resident-data and outcome-cache storage. In machine words, persistent space is $O(S_{\mathsf{data}} + |\mathcal{I}_e| + |\mathcal{T}_e| + |\mathcal{I}_{\mathsf{mv}}| + |\pi^\star| + S_{\mathsf{cache}})$, including delta state in the index terms. Query-local space is $O(d_p + H + |S_{\mathsf{reg}}| + k + W_{\mathsf{inf}} + W_{\mathsf{local}} + W_{\mathsf{page}})$, where d_p stores the projected query and the workspace terms cover inference, local frontiers, and candidate pages.

Append cost. Beyond record storage and ANN insertion, each append requires $O(dd_p + d_p h_\Delta)$ for projection and delta-tree path updates, where h_Δ is the delta-tree height. Preparation and refresh costs are accounted for separately. Appendix D in [34] gives the full derivations.

V. Experimental Study

We evaluated MIFA on vector–scalar hybrid datasets with missing scalar attributes along four dimensions: (1) answer quality under controlled missingness and diverse predicate workloads; (2) inference/search efficiency and scalability; (3) the contributions of hierarchical contexts, serving structures, and inference–search coordination; and (4) exact and approximate execution, as well as incremental maintenance.

Dataset	Pred.	$\lvert D_{\mathsf{tr}}\rvert$	$\lvert D\rvert$	$\lvert Q_{\mathsf{test}}\rvert$	d	$\lvert\bar{\mathcal{A}}\rvert$
WANDS [37]	C+N	15,873	6,349	240	1,024	2
Books [38], [39]	C+N	4,000	1,600	68	1,024	3
MovieLens [40], [41]	C+N	6,606	2,642	300	1,024	4
Yelp [42]	C+N	29,725	11,890	300	1,024	4
H&M [43]	C	52,563	21,025	600	1,024	1
Amazon-multi [38]	C	13,397	5,359	300	1,024	2
AppReviews [44]	N	144,033	57,613	600	1,024	2
Amazon-C4 [38], [39]	C	30,000	12,000	1,000	1,024	1
ArXiv [45]	N	50,000	20,000	1,000	4,096	2
ArXiv-large [45]	N	50,000	2,735,264	1,000	128	2
Books-1M	N/C/C+N	4,000	1,000,000	64	128	3

TABLE I: The tested datasets. C, N, and C+N denote categorical, numerical, and mixed categorical–numerical predicates.

A. Experimental Setting

Datasets and workloads. We evaluated ten real-world vector–scalar hybrid workloads spanning mixed (C+N), categorical (C), and numerical (N) predicates, and a controlled Books-1M scale-up. Table I summarizes dataset sizes, embedding dimensions, and predicate types; the natural-missingness datasets are listed in Appendix E-A (Table VIII). See [34] for details on dataset/query construction and configuration choices.

Data separation. We split the complete relation D^*_{full} by entity into disjoint partitions D^*_{tr}, D^*_{val}, D^*_{cal} and D^*_{test}, and apply the same predeclared masking mechanism to each partition independently: $D_x = \mathrm{Mask}(D^*_x)$. The masked test partition forms the searchable relation $D = D_{\mathsf{test}}$. MIFA uses D_{tr} for template discovery and support construction, D_{val} to select $(\Gamma^\star, \pi^\star)$ and refit $\pi^\star$, and D_{cal} only to select $\theta^\star$. All methods receive the same masked partitions and development data, and any pre-mask supervision. Hidden values never serve as evidence or tuple features.

Missingness. Let ρ be the incomplete-tuple rate, m the number of designated query-relevant attributes, and w the number of masked cells per incomplete tuple. By default, $w = 1$ and $m = 2$ when possible, with one numerical and one categorical attribute for mixed workloads; single-attribute workloads use $m = 1$. We evaluate three masking mechanisms: MCAR samples incomplete tuples uniformly; *historical candidate exposure* (HCE) assigns a β_{exp} fraction of the masking budget to tuples that frequently appear in historical embedding-only top-L lists; and *query-answer exposure* (QAE) assigns an α_{ans} fraction to complete-data answer tuples. By default, we use $\rho = 20\%$, $w = 1$, $\beta_{\mathsf{exp}} = 0.5$, and $\alpha_{\mathsf{ans}} = 1$. All methods use identical masks; see Appendix E-A in [34] for more details.

Queries. We use native benchmark queries when available; otherwise, we construct numerical, categorical, and mixed predicates from training-set quantiles and frequencies. Training queries form $\mathcal{Q}_{\mathsf{hist}}$ and are shared by all workload-aware methods; all methods use the same test queries. See Appendix E-A in [34] for the full construction procedure.

Baselines. We compare MIFA with two classes of baselines.

(1) Missing-value handling. (a) Drop treats unresolved tuples as ineligible, whereas (b) MeanMode imputes missing numerical and categorical values using training-set means and modes, respectively. UnknownPass (UP) accepts tuples whose predicates evaluate to TRUE or UNKNOWN, and then performs exact top-k search. We further evaluate (c) MICE [27], (d) MissForest [28], (e) HoloClean [29], and (f) Raha+Baran [30], [31]. (g) We implement QueryCls, a learned baseline that trains a query-conditioned classifier on pseudo-missing tuple–predicate pairs using observed scalar values, masks, projected embeddings, and predicate encodings. To isolate missing-value handling, we follow each method with exact filtered top-k search over its repaired values or eligibility decisions.

(2) Filtered vector search. (a) FAVOR [15] and (b) SIEVE [7] support all predicate types; (c) iRangeGraph [16] supports numerical ranges; and (d) UNG [6] supports categorical labels. We extend UNG to numerical ranges by mapping values and ranges to labels and label sets, respectively. Each workload includes only methods that support its predicate type; other combinations are reported as N/A. Each applicable static backend is paired with Drop and HoloClean. For numerical-range workloads, we additionally include (e) DSG [17] and (f) WoW [18].

Fairness and metrics. All methods share data splits, masks, and test queries, and we never use test data for tuning. Applicable filtered-ANN backends are tuned to the same validation search-quality target. We tune all baselines over method-specific parameter grids and select their best-performing configurations on held-out data. For QueryCls, we search 101 decision thresholds; for filtered-ANN backends, we tune search parameters to minimize latency subject to the common validation search-quality target (see Appendix E-A in [34] for details). Repair methods use only masked observations; Raha+Baran uses its default annotation budget of 20 labeled training tuples. MIFA uses exact mode unless varied, and we verify its exact outputs against brute-force top-k over the same predicted-eligible set. Our primary metric is $F_1@k$ against the complete-data answers $\mathsf{Ans}^*(q, D^*_{\mathsf{test}})$; we also report eligibility diagnostics and applicable offline, materialization, online, and amortized costs.

Parameters and implementation. By default, we set $k = 10$, $\epsilon = 0$, and $L = 10k$; missingness defaults are given above. We predeclare $\Theta = \{0, 0.01, \ldots, 0.99, 1\}$ and choose $\theta^\star$ on D_{cal} by minimizing the query-level objective $1 - F_1@k$. We use HNSW [35] for ANN seeding and run five repetitions on an Intel Xeon Gold 5320 server (2.20 GHz) with 504 GB RAM and two NVIDIA GeForce RTX 4090 GPUs. Within each repetition, all methods share the same masks and queries.

B. Exp-1: Effectiveness

Overall effectiveness. We evaluated answer recovery against Drop, MeanMode, MICE, MissForest, HoloClean, Raha+Baran and QueryCls. MIFA runs in exact mode, and each baseline is followed by exact filtered top-k search. Table II shows that MIFA achieves the best results on all seven HCE workloads and under natural missingness. See Appendix E-B in [34] for the comparison with lazy mean/mode imputation. We further evaluate QAE as an oracle-controlled stress mechanism. On Books under QAE,

Dataset	Drop	Mean	MICE	MF	HC	R+B	QCls	UP	MIFA
(A) Historical candidate exposure (HCE; $\beta_{\text{exp}} = 0.5$)									
WANDS	0.786	0.818	0.813	0.833	0.788	0.773	0.851	0.104	**0.910**
Books	0.772	0.762	0.694	0.832	0.776	0.832	0.775	0.356	**0.895**
MovieLens	0.925	0.873	0.889	0.862	0.901	0.920	0.932	0.629	**0.934**
H&M	0.768	0.775	0.769	0.928	0.927	0.835	0.907	0.879	**0.970**
Yelp	0.827	0.791	0.823	0.951	0.892	0.936	0.890	0.460	**0.954**
Amazon-multi	0.850	0.533	0.850	0.891	0.885	0.847	0.889	0.199	**0.917**
AppReviews	0.718	0.934	0.927	0.929	0.724	0.877	0.951	0.916	**0.953**
Average	0.807	0.784	0.823	0.889	0.842	0.860	0.885	0.506	**0.933**
(B) Natural missingness									
OpenCatalog [10]	0.933	0.944	0.939	0.939	0.939	0.944	0.972	0.972	**0.978**
Steam [46]($w = 2$)	0.780	0.773	0.780	0.840	0.793	0.800	0.827	0.667	**0.853**
Funds [10]($w = 2$)	0.769	0.783	0.749	0.757	0.769	0.783	0.769	0.737	**0.796**

TABLE II: Overall F_1@10 with $k = 10$. (A) reports controlled HCE at the default setting ($\rho = 20\%$, $w = 1$, $\beta_{\text{exp}} = 0.5$); (B) reports evaluation under natural missingness (dataset-specific missingness rates). Mean/MF/HC/R+B/QCls denote MeanMode/MissForest/HoloClean/Raha+Baran/QueryCls; UP denotes UnknownPass; bold indicates the best result.

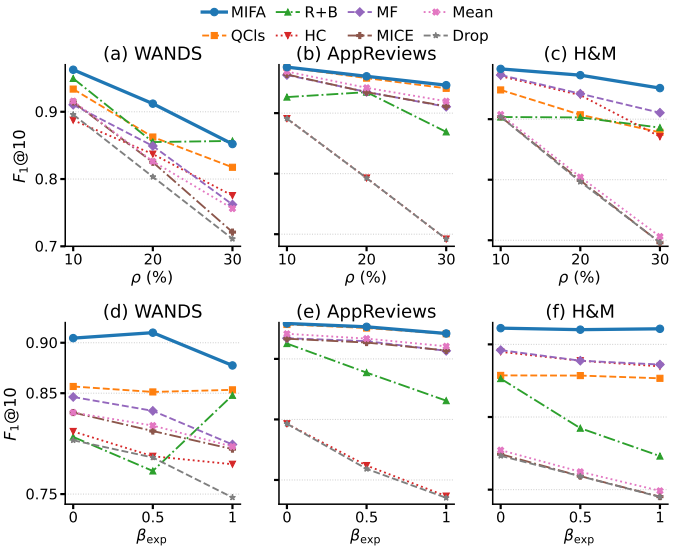


Fig. 4: Varying missingness with ρ and β_{exp}.

QueryCls rejects 52.3% of truly eligible pairs (1.7% for MIFA), and HoloClean leaves 169/320 targeted missing cells unresolved. The complete MCAR/HCE/QAE results, additional diagnostics, and detailed evidence for the QAE results on Books are reported in Appendix E-B in [34].

Varying missingness. We evaluated robustness as missingness becomes more prevalent (MCAR ρ sweep) or more concentrated among historically competitive candidates (HCE β_{exp} sweep), with other settings fixed. Under MCAR, we vary $\rho \in \{10\%, 20\%, 30\%\}$ with $w = 1$. Under HCE, we fix (ρ, w) and vary $\beta_{\text{exp}} \in \{0, 0.5, 1\}$ under the same incomplete-tuple budget, shifting where missingness is placed. Figure 4 shows that increasing ρ from 10% to 30% reduces MIFA's average F_1 by 5.7 points (0.974→0.917), compared with a 7.4-point drop for QueryCls, the strongest baseline (0.953→0.879). Across the β_{exp} sweep, MIFA maintains an average F_1 of 0.930–0.945, exceeding the strongest baseline by 3.1–4.1 points.

Predicate selectivity. We tested predicate selectivity by stratifying queries, with the complete experimental results reported

Method	Handler	F_1@10 ↑	Eng. Rec. ↑	P95 (ms) ↓	QPS ↑	Index (MiB) ↓
FAVOR	Drop	0.568	0.841	155.361	7.3	767.0
FAVOR	HC	0.568	0.841	127.984	8.9	767.0
SIEVE	Drop	0.662	**1.000**	41.121	36.1	21110.0
SIEVE	HC	0.662	**1.000**	41.600	35.7	33067.3
IRG	Drop	0.661	0.996	2.267	525.8	1800.5
IRG	HC	0.660	0.996	2.069	549.7	1800.4
UNG	Drop	0.675	0.958	204.283	16.1	3494.2
UNG	HC	0.696	0.959	205.710	16.0	3694.6
DSG	Drop	0.662	1.000	7.081	225.8	5739.4
DSG	HC	0.662	1.000	7.959	155.1	5739.4
WoW	Drop	0.661	0.999	8.853	150.4	2183.5
WoW	HC	0.661	0.999	9.447	138.3	2183.5
MIFA	Native	**0.799**	**1.000**	**1.090**	**1108.6**	**257.7**

TABLE III: End-to-end comparison on `ArXiv-large` at $d = 128$ under HCE ($\rho = 20\%$, $\beta_{\text{exp}} = 0.5$). IRG for iRangeGraph, HC for HoloClean. We report core online metrics; detailed statistics and breakdowns are in Appendix E-C of [34].

in Tables XIV and XV of [34]. On WANDS, MIFA achieves F_1@10 of 0.859, 0.964, and 0.908 in the low, medium, and high selectivity strata, respectively. Its advantage is not uniform: in the high selectivity stratum of Amazon-multi, MIFA obtains 0.889 while QueryCls reaches 0.957. Thus, average improvements do not imply gains in every stratum. Overall, MIFA achieves strong answer quality across different datasets and predicate selectivities.

We also evaluated missingness patterns, predicate-workload sweeps, selectivity diagnostics, and eligibility-threshold sensitivity; see Appendix E-B in [34] for details.

C. Exp-2: Efficiency

Efficiency. We evaluated end-to-end efficiency on a 1M-row searchable subset of ArXiv-large (2.76M rows) under default HCE. We compare MIFA with all applicable filtered-ANN baselines, each paired with Drop and HoloClean.

As shown in Table III, MIFA achieves 1.090 ms P95 latency and 1108.6 queries per second (QPS), delivering a 1.90× latency speedup and 2.02× the throughput of the fastest external CPU configuration, iRangeGraph +HoloClean. These gains do not sacrifice quality: MIFA achieves F_1@10 = 0.799. Its 257.7-MiB auxiliary index is smaller than the external indexes and is reported alongside the online metrics.

Execution breakdown. We profiled MIFA's execution stages and the effects of eligibility caching. Cohort-level cold-cache resets reveal workload-dependent first-touch costs: on WANDS, mean online latency increases from 1.444 to 6.143 ms, whereas steady-state reuse amortizes eligibility inference. With 153 distinct predicates on AppReviews, P95 latency increases by only 11.0% relative to full predicate reuse. See Appendix E-C in [34] for detailed profiling and caching results, including the stage runtimes in Table XIX.

Preparation cost. We measured the full offline preparation cost. On WANDS, AppReviews, and H&M, MIFA takes 41.74, 37.47, and 174.50 s, respectively. Structure construction takes only 4.58–8.68 s; joint selection and final refitting

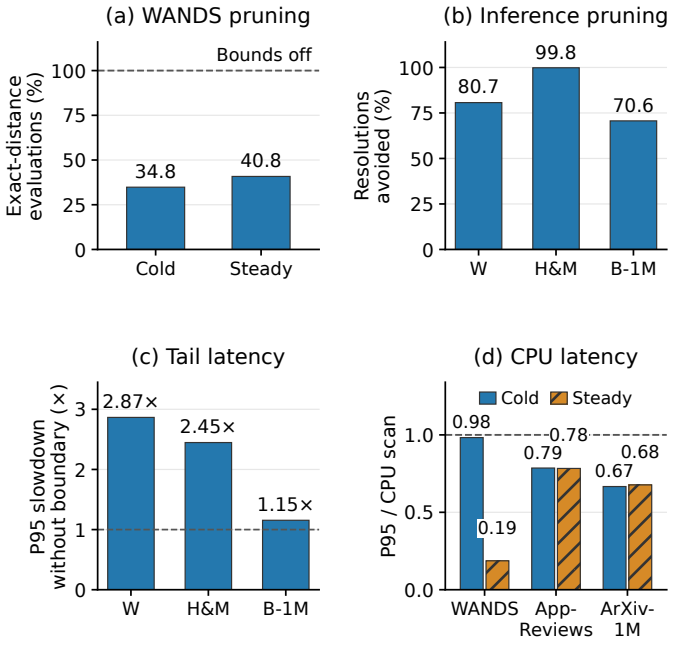


Fig. 5: Pruning power of MIFA and its impact on inference work and tail latency.

Dataset	Execution	P95 (ms)	D2H (KiB/query)	Heap (KiB)
WANDS	Dense output	1.016	24.801	252.885
	MIFA compact	**1.308**	**14.004**	**90.349**
ArXiv-1M	Dense output	28.115	3906.250	36168.653
	MIFA compact	**2.150**	**14.004**	**91.225**

TABLE IV: GPU execution with dense output and compact candidate pages. D2H reports tensor payload per query; heap reports the tracked peak host allocation during a query.

dominate. Over an illustrative 10,000-query window per construction or refresh, the amortized costs are 4.17, 3.75, and 17.45 ms/query. Preparation can exceed static baseline costs: SIEVE +HoloClean takes 43.299 s on H&M. Thus, the overall cost trade-off depends on the queries served between refreshes. See Tables XXII–XXIV in [34] for more experimental results.

Pruning power. We evaluated the pruning power of MIFA's unified best-first executor, which uses geometric bounds and an evolving top-k boundary (Figure 5). On WANDS, geometric bounds reduce exact-distance evaluations by 65.2% with per-request cold resets and 59.2% in steady state, relative to disabling them (panel (a)). Screening unresolved tuples against the top-k boundary avoids 70.6–99.8% of eager statistical resolutions; disabling it raises P95 latency to 1.15–2.87× that obtained with screening enabled (panels (b–c)). Panel (d) reports latency relative to a vectorized CPU scan. These gains preserve the same predicted-eligible top-k under a fixed eligibility rule and exact execution; answer-quality gains come from eligibility inference.

Physical execution. We evaluated whether compact candidate pages reduce data movement, host-memory allocation, and query latency compared with dense output under the same eligibility rule (Table IV). Compact pages reduce device-to-host payload by 43.5–99.6% and tracked peak host allocation by 64.3–99.7%. On ArXiv-1M, P95 latency decreases by 92.4% (28.115→2.150 ms). On WANDS, payload and host allocation decrease, but latency increases from 1.016 to 1.308 ms. Thus, compact pages reduce transfer and materialization costs, while their latency benefits depend on the workload.

Corpus size	MIFA CPU P95 (ms)	Fastest ext. CPU baseline	Ext. P95 (ms)	Speedup	MIFA aux. index (MiB)
20K	0.237	SIEVE +HoloClean	0.396	1.67×	59.4
100K	0.343	DSG +HoloClean	1.344	3.92×	83.0
1M	1.090	iRangeGraph +HoloClean	2.069	1.90×	257.7

TABLE V: Corpus-size scalability on nested ArXiv-large prefixes at $d = 128$ under HCE. Baselines use HoloClean; the fastest baseline varies by scale. Full per-method CPU/GPU results are in Appendix E-C.

Scalability. We evaluated scalability on nested prefixes of ArXiv-Large with 20K, 100K, and 1M rows under HCE. Table V shows that MIFA-CPU achieves P95 latency speedups of 1.67×, 3.92×, and 1.90×, respectively, over the fastest external CPU baseline paired with HoloClean at each scale. Its auxiliary index grows from 59.4 to 257.7 MiB. See Appendix E-C in [34] for complete CPU/GPU results by method and scale, and additional scalability studies.

D. Exp-3: Ablation Study

Hierarchical location-aware contexts. We evaluated hierarchical backoff and embedding locality with two variants. LeafOnly uses the same evidence as MIFA, but abstains when leaf support is insufficient. ScalarOnly removes embedding-local restrictions from the support templates.

Figure 6(a–b) shows that backoff is most beneficial when leaf support is sparse. On WANDS and ArXiv, MIFA improves F_1@10 over LeafOnly by 11.50 and 4.05 percentage points, respectively; LeafOnly resolves only 42.0% and 66.3% of unresolved cases on these workloads. On Amazon-C4, all decisions use leaf contexts, so the two methods achieve the same F_1@10. Removing embedding locality with ScalarOnly lowers F_1@10 on all three workloads by up to 8.25 percentage points. Thus, hierarchical backoff mitigates sparse local support, while embedding locality helps select relevant supporting records.

Calibration sensitivity. We assessed calibration stability by re-selecting θ using 25%, 50%, and 100% of the calibration data over five repetitions, with the inference policy fixed. Across the three HCE workloads, test F_1@10 remains within 1.38 percentage points of the full-calibration result (Figure 6(f)). We also compared the calibration-selected threshold θ_{cal} with the diagnostic test-optimal threshold $\theta^{\star}_{\mathsf{test}}$ under a fixed design. On WANDS, $\theta_{\mathsf{cal}} = 0.40$ and $\theta^{\star}_{\mathsf{test}} = 0.53$ yield a test F_1 regret of 0.014 (1.4 percentage points). On Amazon-multi, the thresholds are 0.66 and 1.00, respectively, with zero regret. Thus, threshold differences need not imply answer-quality losses when F_1 exhibits a plateau. The test-optimal threshold is used

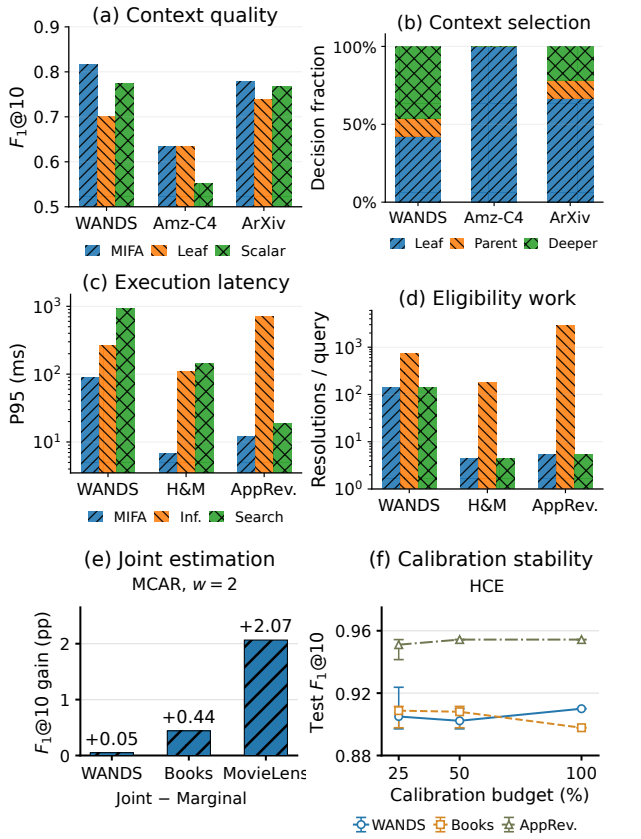


Fig. 6: Hierarchical-context and execution-order ablations, joint estimation, and calibration stability. In (f), error bars show the range across five repetitions.

Dataset	MIFA	$-\mathcal{I}_e$	$-\mathcal{T}_e$	$-\mathcal{I}_{\text{mv}}$	$-\mathcal{J}$
WANDS	0.816 / 54.65	0.816 / 48.73	0.775 / 48.61	0.816 / 332.63	0.775 / 129.15
Amazon-C4	0.634 / 6.91	0.634 / 5.99	0.552 / 3.83	0.634 / 21.29	0.552 / 3.25
ArXiv	0.779 / 20.88	0.779 / 20.30	0.769 / 6.97	0.779 / 949.31	0.769 / 29.16
AppReviews	0.929 / 5.27	0.929 / 3.15	0.884 / 27.26	0.929 / 31.25	0.884 / 21.18

TABLE VI: Serving-structure ablation across four workloads. Each entry reports F_1@10/P95 latency (ms). For WANDS only, we use HCE with $(\rho, \beta_{\text{exp}}) = (30\%, 1)$. MIFA$_{-\mathcal{I}_e}$ and MIFA$_{-\mathcal{I}_{\text{mv}}}$ preserve MIFA's eligibility policy, whereas MIFA$_{-\mathcal{T}_e}$ and MIFA$_{-\mathcal{J}}$ use ScalarOnly inference.

only for post-hoc diagnostics, not deployment selection. See Figure 7 and Table XVII in [34] for more detailed diagnostics.

Joint predicate estimation. We compared joint estimation with the product of marginal estimates, using the same supporting records and separately calibrated thresholds. As shown in Figure 6(e), under MCAR with $w = 2$, joint estimation improves F_1@10 by 0.05, 0.44, and 2.07 percentage points on WANDS, Books, and MovieLens, respectively. These gains highlight the benefit of MIFA's ability to account for dependencies among unresolved predicates when multiple attributes are missing.

Serving-structure ablation. We ablated the serving structures $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$. MIFA$_{-\mathcal{I}_e}$ omits ANN seeding and relies on tree traversal; MIFA$_{-\mathcal{T}_e}$ scans the searchable relation using ScalarOnly; MIFA$_{-\mathcal{I}_{\text{mv}}}$ scans D_{tr} to retrieve the same supporting records; and MIFA$_{-\mathcal{J}}$ scans both relations using scalar-only inference. Only MIFA$_{-\mathcal{I}_e}$ and MIFA$_{-\mathcal{I}_{\text{mv}}}$ preserve MIFA's eligibility policy. All variants use the same fused CPU backend with the hard-pass direct-top-k shortcut disabled.

Table VI shows that replacing $\mathcal{I}_{\text{mv}}$ with scans preserves F_1 but incurs a 3.08–45.46× P95 latency penalty. Removing $\mathcal{T}_e$ eliminates location-aware inference and lowers F_1 by up to 8.25 percentage points. Omitting $\mathcal{I}_e$ preserves quality while reducing latency. These results highlight the benefits of MIFA's shared support index for execution efficiency and its hierarchical inference contexts for answer quality.

Execution-order ablation. We isolated scheduling effects by comparing MIFA with InferenceFirst and SearchFirst under the same fixed $(\pi^\star, \theta^\star)$, supporting records, and exact search semantics. InferenceFirst resolves eligibility before search, SearchFirst sorts all exact distances before resolving eligibility, and MIFA interleaves search and inference. We measured P95 latency and the number of statistical resolutions.

Figure 6(c–d) shows that all three methods achieve identical F_1, while MIFA delivers 2.96–58.84× and 1.55–21.24× P95 speedups over InferenceFirst and SearchFirst, respectively. MIFA avoids 81.5–99.8% of InferenceFirst's statistical resolutions. SearchFirst performs similar eligibility work but first constructs the complete distance order. By tightening the top-k boundary and deferring inference, MIFA avoids unnecessary eligibility evaluations and full distance sorting.

E. Exp-4: Approximation and Maintenance

Approximate execution. We evaluated approximate execution with early candidate resolution to initialize the top-k boundary and relaxed stopping, keeping the eligibility policy and threshold fixed. On H&M with $\epsilon = 0.005$, P95 latency decreases from 75.79 to 22.19 ms (3.42× speedup), while complete-data F_1 decreases from 0.979 to 0.975, a loss of 0.4 percentage points. The maximum observed ratio of the returned kth distance to its exact-mode counterpart is 1.0046, below $1 + \epsilon = 1.005$. Recall against the exact-mode result set is 0.995. The distance-boundary guarantee permits changes in result membership and does not directly bound complete-data F_1. This execution configuration reduces latency while retaining high answer quality. See Table XXVII in [34] for additional exact/approximate comparisons.

Incremental maintenance. We evaluated incremental maintenance by building an initial version from the earliest 60% of the data and appending batches until 80% and 100% of the full relation are searchable. Incremental updates index appended tuples while reusing the current version's fixed base inference contexts and support. On AppReviews, incremental maintenance reduces final-batch publication time T_{visible} from 5.41 to 0.53 s and cumulative maintenance time from 15.78 to 5.88 s compared with restart. Online P95 latency is 1.76 ms, compared with 1.03 ms for restart. See Figure 10 and Table XXVIII in [34] for detailed results.

VI. Conclusion

We formalize *incomplete filtered ANN search* and present MIFA, which infers predicate eligibility without reconstructing missing values. MIFA combines hard decisions from observed values and validated constraints with context-adaptive estimation for unresolved tuples, using on-demand inference in distance-ordered execution with safe pruning. We establish hard-decision soundness, exact execution over the predicted-eligible set, and a $(1+\epsilon)$ distance guarantee for approximate execution; experiments show gains in $F_1@k$ and efficiency.

VII. AI-Generated Content Acknowledgement

The research ideas and code were not generated by artificial intelligence. AI tools were used solely to improve the clarity and readability of the text.

References

[1] Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki, "Filtered vector search: State-of-the-art and research opportunities," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5488–5492, 2025.

[2] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, Y. Zhang, N. Mahapatro, P. Srinivasan *et al.*, "Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 3406–3416.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[4] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–27, 2024.

[5] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, "UNIFY: Unified index for range filtered approximate nearest neighbors search," *Proceedings of the VLDB Endowment*, vol. 18, no. 4, pp. 1118–1130, 2024.

[6] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," *Proceedings of the ACM on Management of Data*, vol. 2, no. 6, pp. 1–27, 2024.

[7] Z. Li, S. Huang, W. Ding, Y. Park, and J. Chen, "SIEVE: Effective filtered vector search with collection of indexes," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 4723–4736, 2025.

[8] L. Libkin, "Sql's three-valued logic and certain answers," *ACM Transactions on Database Systems (TODS)*, vol. 41, no. 1, pp. 1–28, 2016.

[9] P. Guagliardo and L. Libkin, "A formal semantics of SQL queries, its validation, and applications," *Proc. VLDB Endow.*, vol. 11, no. 1, pp. 27–39, 2017.

[10] Open Catalogue, "Open Catalogue: Product data," https://github.com/sedhuait/oc-data, accessed: 2026-10.

[11] J. Cambronero, J. K. Feser, M. J. Smith, and S. Madden, "Query optimization for dynamic imputation," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1310–1321, 2017.

[12] Y. Lin and S. Mehrotra, "ZIP: Lazy imputation during query processing," *Proceedings of the VLDB Endowment*, vol. 17, no. 1, pp. 28–40, 2023.

[13] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems: J. jie et al." *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024.

[14] G. Sehgal and S. Salihoglu, "Navix: A native vector index design for graph dbmss with robust predicate-agnostic search performance," *arXiv preprint arXiv:2506.23397*, 2025.

[15] J. Song, Y. Liu, G. Hu, Z. Xie, M. Yang, B. C. Ooi, and K. Zhou, "FAVOR: Efficient filter-agnostic vector ANNS based on selectivity-aware exclusion distances," *Proceedings of the ACM on Management of Data*, vol. 4, no. 3, pp. 1–25, 2026.

[16] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, "iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search," *Proceedings of the ACM on Management of Data*, vol. 2, no. 6, pp. 1–26, 2024.

[17] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic range-filtering approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 18, no. 10, pp. 3256–3268, 2025.

[18] Z. Wang, J. Zhang, and W. Hu, "WoW: A window-to-window incremental index for range-filtering approximate nearest neighbor search," *Proceedings of the ACM on Management of Data*, vol. 3, no. 6, pp. 1–27, 2025.

[19] J. Zhu, J. Yuan, K. Yang, X. Chen, S. Yu, H. Lv, Y. Li, and B. Zheng, "An experimental evaluation of hybrid querying on vectors," *Proceedings of the VLDB Endowment*, vol. 19, no. 2, pp. 183–195, 2025.

[20] T. Imieliński and W. Lipski, Jr., "Incomplete information in relational databases," *J. ACM*, vol. 31, no. 4, pp. 761–791, 1984.

[21] N. Dalvi and D. Suciu, "Efficient query evaluation on probabilistic databases," in *Proceedings of the 30th International Conference on Very Large Data Bases*, 2004, pp. 864–875.

[22] G. Wolf, H. Khatri, B. Chokshi, J. Fan, Y. Chen, and S. Kambhampati, "Query processing over incomplete autonomous databases." in *VLDB*, vol. 7, 2007, pp. 651–662.

[23] A. Ukkonen, "Approximate Top-k retrieval from hidden relations," 2010. [Online]. Available: https://arxiv.org/abs/1008.5057

[24] K. C.-C. Chang and S.-w. Hwang, "Minimal probing: Supporting expensive predicates for Top-k queries," in *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '02. Association for Computing Machinery, 2002, pp. 346–357.

[25] G. R. Hjaltason and H. Samet, "Distance browsing in spatial databases," *ACM Transactions on Database Systems*, vol. 24, no. 2, pp. 265–318, 1999.

[26] G. Beskales, M. A. Soliman, and I. F. Ilyas, "Efficient search for the top-k probable nearest neighbors in uncertain databases," *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 326–339, 2008.

[27] S. van Buuren and K. Groothuis-Oudshoorn, "mice: Multivariate imputation by chained equations in R," *Journal of Statistical Software*, vol. 45, no. 3, pp. 1–67, 2011.

[28] D. J. Stekhoven and P. Bühlmann, "Missforest—non-parametric missing value imputation for mixed-type data," *Bioinformatics*, vol. 28, no. 1, pp. 112–118, 2012.

[29] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1190–1201, 2017.

[30] M. Mahdavi, Z. Abedjan, R. C. Fernandez, S. Madden, M. Ouzzani, M. Stonebraker, and N. Tang, "Raha: A configuration-free error detection system," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 865–882.

[31] M. Mahdavi and Z. Abedjan, "Baran: Effective error correction via a unified context representation and transfer learning," *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1948–1961, 2020.

[32] W. Fan, F. Geerts, J. Li, and M. Xiong, "Discovering conditional functional dependencies," *IEEE Transactions on Knowledge and Data Engineering*, vol. 23, no. 5, pp. 683–698, 2011.

[33] S. Song and L. Chen, "Differential dependencies: Reasoning and discovery," *ACM Transactions on Database Systems (TODS)*, vol. 36, no. 3, pp. 1–41, 2011.

[34] "Code, datasets and full version," 2026, https://anonymous.4open.science/r/MIFA_Release-68D6.

[35] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[36] C. Yang, Y. Luo, C. Cui, J. Fan, C. Chai, and N. Tang, "Data imputation with limited data redundancy using data lakes," *Proceedings of the VLDB Endowment*, vol. 18, no. 10, pp. 3354–3367, 2025.

[37] Y. Chen, S. Liu, Z. Liu, W. Sun, L. Baltrunas, and B. Schroeder, "WANDS: Dataset for product search relevance assessment," in *Advances in Information Retrieval: 44th European Conference on IR Research, ECIR 2022, Proceedings, Part I*, ser. Lecture Notes in Computer Science, vol. 13185. Springer, 2022, pp. 128–141.

[38] McAuley Lab, "Amazon reviews 2023," https://amazon-reviews-2023.github.io/, 2023.

[39] M. Lab, "Amazon-c4," https://huggingface.co/datasets/McAuley-Lab/Amazon-C4, 2024.

[40] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, pp. 1–19, 2015.

[41] J. Vig, S. Sen, and J. Riedl, "The tag genome: Encoding community knowledge to support novel interaction," *ACM Transactions on Interactive Intelligent Systems*, vol. 2, no. 3, pp. 1–44, 2012.

[42] Yelp Inc., "Yelp open dataset," https://business.yelp.com/data/resources/open-dataset/, dataset; accessed 2026-06-18.

[43] Qdrant Team, "ANN filtered retrieval datasets," https://github.com/qdrant/ann-filtering-benchmark-datasets, 2023, filtered vector-similarity benchmark datasets, including the H&M clothing workload; accessed 2026-08-20.

[44] G. Kang, Z. Ge, J. Hu, X. Zhang, L. Wang, and J. Zhan, "BigVectorBench: Heterogeneous data embedding and compound queries are essential in evaluating vector databases," *Proceedings of the VLDB Endowment*, vol. 18, no. 5, pp. 1536–1550, 2025.

[45] P. Iff, P. Brügger, M. Chrapek, D. Kochergin, M. Besta, and T. Hoefler, "Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors," in *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2026, pp. 610–622.

[46] leinstay, "Steam Game Database," https://github.com/leinstay/steamdb, dataset snapshot: 2026-10-22.

APPENDIX A
VALIDATION-BASED JOINT DESIGN SELECTION

This appendix details the offline procedure in Section IV-B. The searchable relation D supplies search contents and bounds; D_{tr} supplies template candidates, the partition topology, and statistical support; D_{val} supplies design-selection and score-map labels; and D_{cal} supplies threshold-selection labels. Validation and calibration tuples never become statistical support. The procedure first selects a policy design, then fits its deployment maps, and finally selects the acceptance threshold.

Target and body enumeration. For each predicate and predeclared missingness shape represented in $\mathcal{Q}_{\mathsf{val}}$, MIFA enumerates the attributes $Y_\phi(t)$ that may remain unresolved after hard evaluation. Duplicate residual sets are represented once. For each candidate target set Y, dependency discovery searches D_{tr} for bodies $X_s \subseteq \bar{\mathcal{A}} \setminus Y$ and scalar matching conditions. Supporting records must have all attributes in $X_s \cup Y$ observed.

For each retained body–target pair, MIFA generates a scalar-only template and a location-aware counterpart $r : X_s, [\mathsf{Loc}(A_{\mathbf{e}})] \rightsquigarrow Y$. It also considers body-free templates ($X_s = \emptyset$) as fallback candidates. Templates with insufficient training support are discarded, yielding the finite set $\mathcal{T}_{\mathsf{cand}}$. These mined dependencies generate statistical templates; hard certification uses the separately supplied constraints Σ_{det}.

Candidate designs. A structural configuration $\Gamma = (d_p, B_{\mathsf{leaf}})$ specifies the projection dimension and training leaf capacity. A candidate policy π specifies an ordered subset of $\mathcal{T}_{\mathsf{cand}}$, applicability and matching rules, minimum-support requirements, context-backoff sequences, smoothing strengths, and score-map candidate pools and fitting rules. The admissible choices and hyperparameter grids $\mathcal{H}_{\mathsf{val}}$ are fixed before candidate evaluation.

For an unresolved pair, the policy selects the first template whose body conditions apply and whose target covers the unresolved attributes. Body-free templates provide fallback when scalar-body templates are unavailable. For the selected template, insufficient local support triggers context backoff; failure to meet the support requirement in every permitted context yields abstention. These rules are fixed throughout candidate evaluation and deployment. Support failure does not trigger selection of a different template; template selection precedes support-based context backoff.

Partition construction and bounds. For each configuration $\Gamma = (d_p, B_{\mathsf{leaf}})$, MIFA learns a PCA projection from D_{tr}:

$$P(\mathbf{e}) = U_{d_p}^{\mathsf{T}}(\mathbf{e} - \boldsymbol{\mu}), \qquad U_{d_p}^{\mathsf{T}} U_{d_p} = I,$$

where $\boldsymbol{\mu}$ is the training mean and the columns of U_{d_p} are the top d_p principal components. The projection uses no whitening or rescaling. MIFA recursively splits projected training tuples at the median along their largest-variance coordinate until each leaf meets the training capacity B_{leaf} or no nonempty split is possible. Fixed tie-breaking rules make the projection and split tests reproducible.

Training and target tuples follow the same fixed split tests. The nodes on a target tuple's leaf-to-root path define nested inference contexts, whose training populations supply statistical support. After selecting $\Gamma^\star$, MIFA routes D through the fixed topology. Leaves store searchable tuples, and each node N stores $\mathsf{MBR}(N)$ enclosing its projected searchable descendants. Inference contexts depend on the split tests, whereas node bounds depend on searchable contents. Empty searchable subtrees are omitted from traversal. No validation or calibration tuple changes the persistent MBRs.

For Euclidean distance, define

$$\mathsf{LB}(\mathbf{q}_e, N) = \min_{\mathbf{z} \in \mathsf{MBR}(N)} \|P(\mathbf{q}_e) - \mathbf{z}\|_2.$$

Because P is non-expansive, every descendant t satisfies

$$\begin{aligned}\mathsf{LB}(\mathbf{q}_e, N) &\le \|P(\mathbf{q}_e) - P(t[A_{\mathbf{e}}])\|_2 \\ &\le \|\mathbf{q}_e - t[A_{\mathbf{e}}]\|_2.\end{aligned}$$

When query and tuple embeddings are ℓ_2-normalized, cosine distance satisfies

$$\begin{aligned}\mathsf{dist}_{\cos}(\mathbf{q}_e, t[A_{\mathbf{e}}]) &= \frac{\|\mathbf{q}_e - t[A_{\mathbf{e}}]\|_2^2}{2} \\ &\ge \frac{\mathsf{LB}(\mathbf{q}_e, N)^2}{2}.\end{aligned}$$

Hence, the cosine lower bound is $\mathsf{LB}_{\cos}(\mathbf{q}_e, N) = \mathsf{LB}(\mathbf{q}_e, N)^2/2$. Using the bound for the query's distance metric, MIFA safely prunes N when its lower bound exceeds the current top-k boundary τ_B.

Hierarchical eligibility estimation. Suppose the policy selects template r and finds sufficient support. Let $N_j, \ldots, N_J$ be its context sequence from the finest supported context N_j to the broadest permitted context N_J. Let C_i contain the records in D_{tr} within N_i that match r's scalar body and have all required body and target attributes observed. The minimum support size $s_{\min}(r) \ge 1$ applies to matching records, rather than to the total context population. Define $n_i = |C_i|$, and let $n_i^+(\phi)$ count records satisfying the entire residual conjunction $\phi_{\mathsf{res}}(t)$ under the current query constants. MIFA computes

$$\begin{aligned}p_{C_J} &= \frac{n_J^+(\phi)}{n_J}, \\ p_{C_i} &= \frac{n_i^+(\phi) + \kappa_r p_{C_{i+1}}}{n_i + \kappa_r}, \qquad j \le i < J, \\ z_r(t, \phi) &= p_{C_j}.\end{aligned}$$

The parameter $\kappa_r \ge 0$ controls shrinkage toward the next broader estimate, which contributes κ_r pseudo-observations. With $\kappa_r = 0$, the estimate is the empirical satisfaction fraction in the selected context. A scalar-only template uses a single global matching population, so its estimate also reduces to an empirical fraction.

The count $n_i^+(\phi)$ evaluates the complete residual conjunction directly, rather than multiplying separate atom-level estimates. The raw estimate is subsequently transformed by the selected template's score map. Failure to obtain sufficient support yields abstention before score-map evaluation.

Pseudo-missing validation instances. MIFA derives labeled instances $v = (t_v, \phi_v, y_v)$ from validation tuples whose complete predicate outcomes are known, where

$$y_v = \mathbf{1}[\phi_v(t_v^\star) = \mathsf{true}].$$

It masks selected observed predicate values to produce a fixed snapshot $\widetilde{D}_{\mathsf{val}}$. Masked-out values are retained only for labels and never used as inference features or support. Natural missingness remains unchanged, and instances with indeterminate complete outcomes are excluded. The masks, queries, and validation instances are shared by all candidate designs.

For each Γ, MIFA learns a temporary partition topology from D_{tr}. Training tuples populate its inference contexts and support index; masked validation tuples populate temporary searchable leaves and MBRs for executor replay. The latter do not contribute statistical support. The resulting admissible structure–policy pairs form the finite candidate set $\mathcal{C}$.

Score-map candidates and fitting. For each template r, the policy fixes a finite candidate pool $\mathcal{G}_r$ containing the identity map $g_{\mathsf{id}}(z) = z$. A possible parametric family is the monotone sigmoid map

$$g_{a,b}(z) = \sigma(a\ \mathrm{logit}(\bar{z}) + b)\,, \qquad \bar{z} = \mathrm{clip}(z, \xi, 1-\xi),$$

where $\sigma(x) = 1/(1+e^{-x})$, $\mathrm{logit}(u) = \log(u/(1-u))$, $a > 0$, and $0 < \xi < 1/2$. Clipping prevents undefined logits, and the positive slope preserves raw-score ordering within each template. Maps adjust scores across templates; their outputs are eligibility scores, without a guarantee of probability calibration. For finite predeclared parameter sets $\mathcal{A}_r$ and $\mathcal{B}_r$, an example pool is

$$\mathcal{G}_r = \{g_{\mathsf{id}}\} \cup \{g_{a,b} : a \in \mathcal{A}_r,\ b \in \mathcal{B}_r\}.$$

The family, parameter grids, and numerical constants are fixed before candidate evaluation.

Let $\mathcal{U}_r$ be the non-abstaining fitting instances assigned to r, with raw estimates $z_r(v)$ and labels y_v. When fitting data are sufficient, MIFA selects a map by grid search:

$$g_r = \operatorname*{arg\,min}_{g \in \mathcal{G}_r} \frac{1}{|\mathcal{U}_r|} \sum_{v \in \mathcal{U}_r} \big(g(z_r(v)) - y_v\big)^2.$$

A fixed candidate order resolves ties, prioritizing the identity map.

The policy also fixes minimum fitting-data requirements, including a predeclared minimum number of distinct fitting units and representation of both eligibility labels. For entity-grouped data, these requirements use entity groups consistently with the outer split. Repeated queries involving the same tuple or group do not increase the distinct-unit counts. If $\mathcal{U}_r$ is empty or fails these requirements, MIFA sets $g_r = g_{\mathsf{id}}$.

This fallback preserves the supported hierarchical estimate rather than introducing an additional abstention. Insufficient training support still yields inference-stage abstention, whereas insufficient map-fitting data yields identity mapping. The candidate pools and fitting-data requirements form part of the fixed policy specification.

Cross-fitted score maps. MIFA partitions unresolved validation instances $\mathcal{V}_{\mathsf{val}}$ into F fixed folds consistent with the outer data split. Instances sharing a tuple or entity remain in the same fold. For a candidate (Γ, π), let $r(v)$ be the template selected for v, and let $z_r(v)$ denote its raw hierarchical estimate when sufficient support exists.

For each template r, let $\mathcal{V}_r$ contain the non-abstaining validation instances assigned to it. For fold f, define the fitting set

$$\mathcal{U}_r^{(-f)} = \{v \in \mathcal{V}_r : f(v) \neq f\},$$

where $f(v)$ is the fold containing v. MIFA applies the grid-search and identity-fallback rules above:

$$g_r^{(-f)} = \mathsf{FitMap}_r\big(\mathcal{U}_r^{(-f)}\big).$$

Both map selection and fitting-data checks use only $\mathcal{U}_r^{(-f)}$.

For a non-abstaining instance v, its cross-fitted score is

$$\widetilde{p}^{\mathsf{cf}}_{\Gamma,\pi}(v) = g_{r(v)}^{(-f(v))}\big(z_{r(v)}(v)\big).$$

Thus, the label of v is excluded from the fitting data used to score it. All raw estimates use supporting records exclusively from D_{tr}; and validation labels fit score maps but never enter the support index.

Predicate-inference quality. For evaluating validation loss only, MIFA assigns score zero to an abstention. During execution, abstention remains a separate outcome and is always treated as ineligible, independently of the threshold. Define

$$\mathsf{InfLoss}(\Gamma, \pi) = \frac{1}{|\mathcal{V}_{\mathsf{val}}|} \sum_{v \in \mathcal{V}_{\mathsf{val}}} \big(\widetilde{p}^{\mathsf{cf}}_{\Gamma,\pi}(v) - y_v\big)^2$$

and

$$\mathsf{InfScore}(\Gamma, \pi) = 1 - \mathsf{InfLoss}(\Gamma, \pi).$$

This evaluates eligibility estimation for unresolved conjunctions. Predeclared coverage requirements additionally control which designs are admissible. Coverage is the fraction of applicable unresolved instances that obtain sufficient support, measured separately for the declared template groups and predicate–missingness cases. Only nonempty evaluation groups enter these checks. Candidate evaluation requires a nonempty validation-instance set; all candidates use the same instances and group definitions.

End-to-end replay cost. Validation selects a design but does not select the deployment threshold. MIFA fixes a finite representative threshold set Θ_{rep} before evaluation and replays exact execution for each $q \in \mathcal{Q}_{\mathsf{val}}$ and $\vartheta \in \Theta_{\mathsf{rep}}$. Each replay searches the masked validation snapshot using the candidate hierarchy and support index. A validation-only ANN seed index is fixed across candidates. Every non-abstaining unresolved instance uses its out-of-fold score map, including identity maps selected by fallback.

For one replay, let $m_{q,\vartheta}$ be the number of node-bound evaluations; $n_{q,\vartheta}$ the number of tuple-identifier examinations; $n^{\mathsf{dist}}_{q,\vartheta}$ and $n^{\mathsf{det}}_{q,\vartheta}$ the numbers of exact-distance and deterministic-

predicate evaluations; $u^{\mathsf{queue}}_{q,\vartheta}$ the number of unresolved tuples inserted into the queue; $u^{\mathsf{infer}}_{q,\vartheta} \le u^{\mathsf{queue}}_{q,\vartheta}$ the number invoking statistical inference; $b_{q,\vartheta}$ the number of buffer-update attempts; and $H_{q,\vartheta}$ the maximum queue size. The replay cost is

$$\begin{aligned} T_{\mathsf{rep}}(q,\vartheta;\Gamma,\pi) = {} & C_{\mathsf{seed}}(q) + C_{\mathsf{proj}}(q;\Gamma) \\ & + m_{q,\vartheta} c_{\mathsf{lb}} \\ & + (m_{q,\vartheta} + u^{\mathsf{queue}}_{q,\vartheta}) c_Q \log(H_{q,\vartheta}+1) \\ & + n_{q,\vartheta} c_{\mathsf{reg}} + n^{\mathsf{dist}}_{q,\vartheta} c_v \\ & + n^{\mathsf{det}}_{q,\vartheta} c_{\mathsf{det}} + u^{\mathsf{infer}}_{q,\vartheta} c_{\mathsf{inf}} \\ & + b_{q,\vartheta} c_B \log(k+1). \end{aligned}$$

Here, C_{seed} covers backend candidate retrieval, and $C_{\mathsf{proj}} = O(dd_p)$ covers the query projection. The constants c_Q, c_B, and c_{reg} account for queue, buffer, and registration operations, respectively; c_v, c_{lb}, and c_{det} account for distances, bounds, and hard checks. Distances already charged to C_{seed} are excluded from $n^{\mathsf{dist}}_{q,\vartheta}$. The inference coefficient c_{inf} covers indexed support retrieval, context backoff, joint counting, shrinkage, and score-map evaluation. Unit costs are estimated by microbenchmarking and reused across candidate evaluation.

The mean replay cost is

$$\overline{T}_{\mathsf{rep}}(\Gamma,\pi) = \frac{\sum_{q\in\mathcal{Q}_{\mathsf{val}}} \sum_{\vartheta\in\Theta_{\mathsf{rep}}} T_{\mathsf{rep}}(q,\vartheta;\Gamma,\pi)}{|\mathcal{Q}_{\mathsf{val}}||\Theta_{\mathsf{rep}}|},$$

and its normalized value is

$$\mathsf{TimeCost}(\Gamma,\pi) = \frac{\overline{T}_{\mathsf{rep}}(\Gamma,\pi)}{\max_{(\Gamma',\pi')\in\mathcal{C}} \overline{T}_{\mathsf{rep}}(\Gamma',\pi')}.$$

If all replay costs are zero, MIFA sets $\mathsf{TimeCost}(\Gamma,\pi) = 0$ for every candidate. Geometric pruning is reflected in the replay operation counts, so no separate pruning reward is added.

Joint selection and final refitting. Given a predeclared $\lambda \ge 0$, MIFA selects

$$\begin{aligned} (\Gamma^\star,\pi^\star) = \arg\max_{(\Gamma,\pi)\in\mathcal{C}} \big[& \mathsf{InfScore}(\Gamma,\pi) \\ & - \lambda\, \mathsf{TimeCost}(\Gamma,\pi)\big]. \end{aligned}$$

If no candidate satisfies the declared coverage requirements, selection reports that no admissible design is available. Otherwise, it resolves selection ties by the lexicographic order of (d_p, B_{leaf}) and then of candidate policies. Larger λ gives greater weight to online cost.

At this point, $\pi^\star$ fixes the templates, applicability and ordering rules, support and shrinkage parameters, backoff sequences, score-map candidate pools, and fitting-data requirements, but not the final map choices. MIFA discards temporary search structures, support indexes, and fold-specific maps.

Under $\Gamma^\star$, MIFA constructs the final topology and split tests from D_{tr}. It routes D through them to populate searchable leaves and MBRs, builds $\mathcal{I}_e$ over the searchable embeddings, and builds $\mathcal{I}_{\mathsf{mv}}$ from the training evidence. It then routes all pseudo-missing validation instances through the final hierarchy and recomputes their raw estimates using support exclusively from D_{tr}.

Let $\mathcal{V}^\star_r$ contain the non-abstaining validation instances assigned to selected template r under this final design. MIFA applies the same fitting rule to obtain

$$g^\star_r = \mathsf{FitMap}_r\big(\mathcal{V}^\star_r\big).$$

This searches the fixed candidate pool when fitting data are sufficient and otherwise returns the identity map. These maps complete $\pi^\star$. The deployed score for a supported pair is

$$\widehat{p}_{\pi^\star}(t,\phi) = g^\star_r\big(z_r(t,\phi)\big).$$

Final refitting does not add validation tuples or labels to the support index.

Calibration and publication. With the serving structures and deployment policy $\pi^\star$, including its score maps, fixed, MIFA evaluates the policy on the masked calibration snapshot D_{cal}. Calibration tuples follow the fixed split tests for context assignment, while supporting records remain restricted to D_{tr}. They enter neither the persistent search structures nor the support index and modify no MBR.

Let $G_1,\dots,G_{m_{\mathsf{cal}}}$ be the nonempty calibration query groups partitioning $\mathcal{Q}_{\mathsf{cal}}$. Queries sharing a source entity or another declared dependence unit remain in the same group. For the predeclared query-level loss $L(q;\pi^\star,\theta,D_{\mathsf{cal}}) \in [0,1]$ (*e.g.*, $1 - F_1@k$), define

$$\ell_\theta(G_i) = \frac{1}{|G_i|} \sum_{q\in G_i} L(q;\pi^\star,\theta,D_{\mathsf{cal}})$$

and

$$\widehat{R}_{\mathsf{cal}}(\theta) = \frac{1}{m_{\mathsf{cal}}} \sum_{i=1}^{m_{\mathsf{cal}}} \ell_\theta(G_i).$$

Thus, queries are averaged within each group and groups receive equal weight. When each group contains one query, this reduces to mean query-level loss.

The finite threshold set Θ and its tie-breaking order are fixed before observing the calibration groups. For each $\theta\in\Theta$, MIFA applies the fixed policy and computes exact top-k answers by scan and ranking over D_{cal}. It selects

$$\theta^\star \in \arg\min_{\theta\in\Theta} \widehat{R}_{\mathsf{cal}}(\theta).$$

During calibration, hidden values in $D^\star_{\mathsf{cal}}$ provide complete-data answers and loss labels only; they are excluded from inference features and statistical support.

Calibration changes only θ, leaving $\Gamma^\star$, the structures, $\pi^\star$, and its score maps fixed.

MIFA publishes D, $\mathcal{I}_e$, $\mathcal{T}_e$, $\mathcal{I}_{\mathsf{mv}}$, $\pi^\star$, and $\theta^\star$ as one read-consistent serving version.

Between refreshes, base and delta trees cover the active searchable relation. Appended tuples use fixed base inference contexts and contribute no statistical support within the current serving version. A refresh rebuilds and atomically publishes a compatible structure–policy version.

APPENDIX B
SHARED SUPPORT-INDEX CONSTRUCTION AND RETRIEVAL

This appendix details the shared support index $\mathcal{I}_{\mathsf{mv}}$ introduced in the offline design phase of the main paper. It organizes observed scalar evidence from the training data across retained templates and hierarchical contexts without materializing a separate support list for every template–node pair.

Index construction. After fixing the partition tree $\mathcal{T}_e$, MIFA builds a single $\mathcal{I}_{\mathsf{mv}}$ over D_{tr} for all retained templates $r : X_s, [\mathsf{Loc}(A_{\mathbf{e}})] \rightsquigarrow Y$. It stores: (1) for each categorical body attribute, postings from observed values to training-tuple identifiers; (2) for each numerical body attribute, postings over buckets induced by the finest retained matching ranges; (3) for each retained target attribute, identifiers of training tuples in which it is observed; and (4) for each training tuple, the depth-first identifier of the leaf to which $\mathcal{T}_e$ routes it. Because depth-first numbering makes the leaves of each subtree contiguous, the last component represents any hierarchical context as a leaf interval. All components are shared across templates and contexts, and only D_{tr} contributes statistical support.

Support retrieval. Given a target tuple t and retained template $r : X_s, [\mathsf{Loc}(A_{\mathbf{e}})] \rightsquigarrow Y$, MIFA instantiates the template's categorical keys and numerical ranges from $t[X_s]$. For each numerical condition, it retrieves the intersecting buckets and checks the exact condition to remove bucket-level false matches. It then intersects the body matches with the identifiers of training tuples whose target attributes Y are observed. Observed-value postings likewise exclude missing body values. If the template contains $\mathsf{Loc}(A_{\mathbf{e}})$, the selected context node N further restricts the result to its contiguous leaf interval. The resulting tuples satisfy the template's scalar and locality conditions and have observed targets. For a body-free template, body matching imposes no restriction; for a scalar-only template, no leaf-interval restriction is applied. The same shared postings retrieve support at broader contexts during backoff, and retained training values determine satisfaction of the current residual conjunction. Thus, the index reuses evidence across templates, contexts, and query constants without storing a separate list for every combination.

APPENDIX C
THEORETICAL GUARANTEES AND PROOFS

Fix a query q, a read-consistent searchable snapshot D, and its compatible serving version $(\mathcal{J}, \pi^\star, \theta^\star)$. Inference contexts, support records, score maps, and the tuple ordering $\preceq_q$ remain fixed throughout execution. The guarantees concern hard decisions and execution under this fixed eligibility rule.

Hard-decision soundness. We prove Theorem 2.

Theorem 2: *Assume that all constraints in Σ_{det} are valid for D^* and that hard evaluation uses nonempty certified domains as described in Section IV-C. For query q, let E_{pass} and E_{fail} be the tuples receiving hard* pass *and hard* fail*, respectively. Then $E_{\mathsf{pass}} \subseteq E^*(q, D^*)$ and $E_{\mathsf{fail}} \cap E^*(q, D^*) = \emptyset$.* □

Proof: Because D^* agrees with D on observed values, evaluation of an observed atom is sound. For a missing attribute A, every applicable valid constraint provides a domain containing $t^*[A]$. Their nonempty intersection $V(t, A)$ therefore satisfies

$$t^*[A] \in V(t, A).$$

If every value in $V(t, A)$ satisfies an atom, the hidden value satisfies it. If no value satisfies the atom, the hidden value violates it. A partially overlapping domain, an empty intersection, or the absence of an applicable constraint yields no hard certification.

A hard pass requires every conjunct to be true through observation or certification. A hard fail requires at least one conjunct to be false through observation or certification. The two set relations follow. □

Threshold monotonicity. Fix the structures, support records, and policy $\pi^\star$, including its score maps. For $\theta_1 \le \theta_2$,

$$\widehat{E}_{\pi^\star,\theta_2}(q, D) \subseteq \widehat{E}_{\pi^\star,\theta_1}(q, D).$$

Proof: Hard-pass tuples are accepted independently of the threshold, while hard-fail tuples and abstentions remain excluded. A supported unresolved tuple is accepted exactly when its fixed score is at least the threshold. Acceptance at θ_2 therefore implies acceptance at θ_1. □

This inclusion concerns predicted eligibility. It does not imply monotonicity of the returned top-k set or answer F_1.

Proof of exact execution in Theorem 1. We prove part (a).

Proof: At each traversal-loop boundary, B contains the best up to k accepted tuples processed in the current query. Buffer updates preserve this property. The registration set records tuples already handled, including safely excluded tuples, and unresolved tuples already scheduled for inference. Registration alone does not imply that eligibility has been evaluated.

Once B becomes full, its worst tuple can only improve under $\preceq_q$, and its distance boundary cannot increase. Suppose a node is pruned when the boundary is τ_s. Every descendant has distance strictly greater than τ_s and therefore cannot improve any subsequent buffer. If an unregistered tuple is excluded directly, it does not rank ahead of the current worst buffered tuple under $\preceq_q$. Because the worst buffered tuple can only improve, that tuple also cannot improve a subsequent buffer. Both pruning rules are therefore safe, including distance ties.

The coverage invariant holds initially because the active tree roots cover all searchable tuples, including unresolved ANN seeds. At each iteration, expanding an internal node replaces it with its children except those safely pruned. Processing a leaf handles each unregistered tuple by applying its eligibility outcome, safely excluding it, or queuing it for statistical inference. A queued unresolved tuple is subsequently evaluated or safely excluded after its competitiveness is rechecked. Thus, at every loop boundary, each tuple not finalized or safely excluded remains represented by a queued tuple or an unexpanded node.

If execution ends with $Q = \emptyset$, every tuple has been

handled or safely excluded. The buffer is the top-k of the fixed predicted-eligible set.

Otherwise, exact termination requires $|B| = k$ and $\lambda_Q > \tau_B$. Suppose an omitted predicted-eligible tuple t precedes the worst tuple in B. It cannot have been processed and omitted by the buffer, or safely excluded, by the preceding arguments. It must therefore be represented by an entry $x_t \in Q$. Validity of the entry's lower bound gives

$$\lambda_Q \le \mathsf{key}(x_t) \le \mathsf{dist}(\mathbf{q}_e, t[A_\mathbf{e}]) \le \tau_B,$$

contradicting the stopping condition. Consequently, $\widehat{\mathsf{Ans}}_{\pi^\star,\theta^\star,0}(q, D) = \mathsf{Ans}_{\pi^\star,\theta^\star}(q, D)$. □

Proof of approximate execution in Theorem 1. We prove part (b).

Proof: Assume that $|\widehat{E}_{\pi^\star,\theta^\star}(q, D)| \ge k$. The buffer is full at termination: early stopping requires a full buffer, while queue exhaustion processes or safely excludes every tuple and cannot leave fewer than k accepted tuples in this case. Since B contains k predicted-eligible tuples,

$$d_k^{\mathsf{pred}} \le \tau_B.$$

If $Q = \emptyset$, the exact-execution argument applies, giving $\tau_B = d_k^{\mathsf{pred}}$. Otherwise,

$$\tau_B \le (1 + \epsilon)\lambda_Q.$$

Every processed predicted-eligible tuple outside B has distance at least τ_B. Every safely excluded tuple also has distance at least the final τ_B, because full-buffer boundaries cannot increase. Every unprocessed predicted-eligible tuple is represented in Q and has distance at least λ_Q.

If $\lambda_Q < \tau_B$, all predicted-eligible tuples with distance below λ_Q must lie in B. Since its worst tuple has distance $\tau_B > \lambda_Q$, at most $k - 1$ predicted-eligible tuples lie below λ_Q. Hence,

$$d_k^{\mathsf{pred}} \ge \lambda_Q$$

and

$$\tau_B \le (1 + \epsilon)\lambda_Q \le (1 + \epsilon)d_k^{\mathsf{pred}}.$$

If $\lambda_Q \ge \tau_B$, no predicted-eligible tuple outside B has distance below τ_B. The worst tuple in B has distance τ_B, so $d_k^{\mathsf{pred}} = \tau_B$, which also proves the claim.

If fewer than k tuples are predicted eligible, the buffer never becomes full. Neither mode can terminate early, and queue exhaustion returns all predicted-eligible tuples. □

Base–delta and physical execution. For base–delta execution, take D to be the active union of base and appended tuples. The two roots provide complete coverage, while the common queue and buffer preserve the same invariants.

The physical implementations in Section IV-D preserve these guarantees when cached outcomes use the same serving version, candidate streams remain resumable, and valid lower bounds cover all unconsumed candidates. Paging and partitioned execution retain a common answer boundary and frontier coverage. These conditions preserve exact-mode results and the approximate distance-boundary guarantee. They do not bound statistical eligibility errors or answer F_1 against the complete-data target. Even with sufficient support, missingness-dependent sampling or distribution shift can affect the inferred eligibility decisions.

APPENDIX D
COMPLEXITY DETAILS

We separate logical query work, persistent storage, query-local workspace, and construction or maintenance costs. All space quantities below are measured in machine words, with scalar values represented by fixed-size codes or values.

Online complexity. Let m_v be the number of node-bound evaluations, including children subsequently pruned; n_v the number of tuple-identifier examinations, counting repeats; n_v^{dist} the number of exact-distance evaluations outside backend candidate retrieval; and n_v^{det} the number of hard evaluations. These counts include processing of ANN seeds and visited leaves.

Let u_q be the number of unresolved tuples inserted into the unified queue, $u_v \le u_q$ the number invoking statistical inference, b_v the number of buffer-update attempts, and H the maximum unified-queue size. Let C_{seed} cover backend candidate retrieval. A distance evaluation is charged either to this backend cost or to n_v^{dist}, without double counting.

Let c_{lb}, c_v, c_{det}, and c_{inf} bound the work of one node-bound evaluation, exact-distance evaluation, hard evaluation, and statistical policy evaluation, respectively. The inference cost includes template selection, support retrieval, context backoff, joint counting, shrinkage, and score-map evaluation.

Assuming expected $O(1)$ registration-set operations, the logical query work is

$$\begin{aligned} O\big(&dd_p + C_{\mathsf{seed}} + m_v c_{\mathsf{lb}} + (m_v + u_q)\log(H + 1) \\ &+ n_v + n_v^{\mathsf{dist}} c_v + n_v^{\mathsf{det}} c_{\mathsf{det}} \\ &+ u_v c_{\mathsf{inf}} + b_v \log(k + 1)\big). \end{aligned}$$

The dd_p term projects the query once, with the result reused for all node bounds. For Euclidean or normalized-cosine search, a node-bound evaluation costs $O(d_p)$ and an exact distance costs $O(d)$. Noncompetitive leaf tuples avoid hard evaluation, while unresolved tuples that become noncompetitive before reaching the queue head avoid statistical inference. These savings depend on the realized operation counts; the worst case may examine all searchable tuples.

Physical execution costs. The preceding expression describes logical work, not parallel wall-clock latency. CPU/GPU implementations additionally perform local candidate selection, stream merging, and candidate-page handling. All distance evaluations must be accounted for, including those whose records are suppressed before reaching the host.

If M_q candidate records (including prefetched records) are emitted to the host, the transfer volume is $O(M_q)$ words for fixed-size records. This bound concerns transfer only; it does not bound the number of distance evaluations. Cached outcomes reduce repeated eligibility work only within the compatible serving version.

Persistent space. Let n be the number of active searchable tuples, $n_{\mathsf{tr}} = |D_{\mathsf{tr}}|$, and M_{tr} the number of indexed observed scalar cells in D_{tr}. Let $N_{\mathcal{T}}$ count all stored context and search-tree nodes, including base and delta state, with shared topology counted once. Its size depends on the training topology as well as the searchable contents.

Let S_{data} be the resident raw-data footprint, including searchable vectors and scalar columns, retained training data, and any stored projected tuple coordinates. Dense searchable data alone uses $O(n(d + |\bar{\mathcal{A}}|))$ words. Let S_e be the additional plug-in ANN storage, s_π the stored policy size, and S_{cache} the outcome-cache footprint. The policy size includes deterministic evidence, templates, applicability and ordering rules, support and shrinkage parameters, backoff sequences, and score maps.

Persistent space for one serving version is

$$O\big(S_{\mathsf{data}} + S_e + n + d_p N_{\mathcal{T}} + dd_p + n_{\mathsf{tr}} + M_{\mathsf{tr}} + s_\pi + S_{\mathsf{cache}}\big).$$

The additional terms cover leaf identifiers, projected MBRs and split metadata, the projection matrix, training-context locations, and shared support-index entries. Outcome-cache storage depends on the tuples and predicates retained in the cache and is not assumed constant.

Shared allocations are counted once. For replicated GPU execution, each allocated data or index copy is counted; for partitioned execution, storage is summed across shards. If old and new versions coexist during refresh, peak storage also includes the retained old version and construction workspace.

Query-local space. Let $r_v = |S_{\mathsf{reg}}|$, where $r_v \le \min\{n, n_v\}$, and let w_{inf} be the maximum temporary workspace for one statistical policy evaluation. For the sequential logical executor, reusable inference workspace gives $O(d_p + H + r_v + k + w_{\mathsf{inf}})$ query-local space.

For physical execution, let W_{local} be the aggregate worker-local frontier and selection workspace, W_{page} the candidate-page and transfer-buffer workspace, and W_{inf} the aggregate workspace of concurrent inference calls. The corresponding space is $O\big(d_p + H + r_v + k + W_{\mathsf{local}} + W_{\mathsf{page}} + W_{\mathsf{inf}}\big)$. The d_p term stores the projected query. The registration set may contain all n tuple identifiers; the analysis therefore does not assume sublinear worst-case query space.

Append costs. Let h_Δ be a root-to-leaf path length in the active delta tree. Beyond raw-record storage and backend-specific ANN insertion, projection and delta-tree maintenance require $O(dd_p + d_p h_\Delta)$ work per append. The first term projects the embedding; the second routes the tuple and updates counts and MBRs along its path. The frozen projection and split tests avoid online repartitioning. This bound excludes periodic refresh.

Offline construction and refresh. Let C_{disc}, C_{joint}, C_{build}, C_{refit}, and C_{cal} be the costs of template discovery, joint design evaluation, serving-structure construction, final policy refitting, and threshold calibration, respectively. With each operation charged to one phase,

$$C_{\mathsf{prep}} = C_{\mathsf{disc}} + C_{\mathsf{joint}} + C_{\mathsf{build}} + C_{\mathsf{refit}} + C_{\mathsf{cal}}.$$

Joint evaluation includes candidate structures, cross-fitted score-map evaluation, and validation replay. Final refitting includes recomputing validation estimates under the final hierarchy and fitting the deployed maps. Calibration includes policy evaluation and exact scan/ranking for the declared threshold candidates.

A refresh additionally merges the active relation and repeats the required offline phases. These costs are separate from online query latency. For a reporting window containing Q queries, initial preparation and subsequent refresh costs contribute $\frac{C_{\mathsf{prep}} + \sum_j C_{\mathsf{refresh},j}}{Q}$ amortized time per query. Tables XXII– XXIV report the corresponding measured components.

Setting	Value or candidate set
Default k	10
Default incomplete-tuple rate ρ	20%
Default missing attributes m	2 when applicable; 1 for native single-attribute workloads
Masked cells per incomplete tuple w	1
Candidate region size L	$10k$
Exposure fraction β_{exp}	0.5; varied over $\{0, 0.5, 1\}$
Oracle answer exposure γ_{ans}	$\{0.25, 0.5, 0.75, 1\}$ (appendix only)
Projection dimension d_p	$\{8, 16\}$
Leaf capacity B_{leaf}	$\{64, 128\}$
Minimum support $s_{\min}$	$\{10, 30, 50\}$
Shrinkage strength κ_r	$\{0, 10, 30\}$
Template and shape coverage	0.80/0.80
Representative thresholds Θ_{rep}	$\{0.25, 0.50, 0.75\}$
Eligibility thresholds Θ	$\{0, 0.01, \ldots, 0.99, 1\}$
ANN seed size	64
Approximation values ϵ	$\{0, 0.0025, 0.005, 0.01, 0.02, 0.05, 0.075, 0.10\}$
Historical-evidence fractions	$\{25\%, 50\%, 75\%, 100\%\}$
Raha+Baran annotation budget	20 labeled training tuples
Queries per dataset	64–1,000 test queries; dataset-specific (Table I)
Worker threads	8; 32 for the optimized Books-1M CPU path
Independent repetitions	5

TABLE VII: Default settings and predeclared candidate grids.

APPENDIX E
ADDITIONAL EXPERIMENTAL DETAILS AND RESULTS

A. Detailed Experimental Configurations

Fixed settings and candidate grids. Table VII reports the default settings and predeclared candidate grids used in our experiments. All candidate sets are fixed before validation. Dataset-specific choices are selected using D_{val} only, after which the deployment policy is refitted and θ is selected using D_{cal} as described in Appendix A. No test query, hidden test value, or test result is used to select any configuration.

The template families, applicability rules, and template/context orders follow Appendix A. Table IX reports the

Dataset	Pred.	$\lvert D_{\mathsf{tr}}\rvert$	$\lvert D\rvert$	$\lvert Q_{\mathsf{test}}\rvert$	d	$\lvert\bar{\mathcal{A}}\rvert$
OpenCatalog [10]	N	989	522	18 768		3
Steam [46]	N	4,796	1,627	15 768		3
Funds [10]	C	4,066	1,389	15 768		4

TABLE VIII: Natural-missingness dataset configurations corresponding to the main-text Table I.

selected partition configuration, retained template order, and calibrated threshold. Its caption gives the common selected support and smoothing settings.

The following details expand the dataset, query, and missingness construction summarized in Section V-A.

Natural-missingness dataset configurations.

Dataset and query construction. WANDS retains its product-search queries and adds categorical and numerical metadata predicates. Books combines Amazon-C4 query embeddings with Amazon Reviews 2023 metadata predicates; MovieLens, Yelp, and Amazon-multi use held-out content queries with their corresponding metadata. ArXiv retains its benchmark query vectors with numerical range/equality filters, while Amazon-C4 retains its categorical filtered workload. AppReviews and H&M retain their native compound-numerical and categorical workloads, respectively, and Books-1M reuses the Books queries. For constructed predicates, attributes are fixed from the schema and training workload, and constants use training quantiles or frequencies.

Data and query separation. We partition the complete relation $D^{\star}_{\mathsf{full}}$ by entity, or chronologically for timestamped datasets, into mutually disjoint partitions $D^{\star}_{\mathsf{tr}}$, $D^{\star}_{\mathsf{val}}$, $D^{\star}_{\mathsf{cal}}$, and $D^{\star}_{\mathsf{test}}$. We independently apply the same predeclared masking mechanism to each partition, obtaining $D_x = \mathsf{Mask}(D^{\star}_x)$ for $x \in \{\mathsf{tr}, \mathsf{val}, \mathsf{cal}, \mathsf{test}\}$. The masked test partition forms the searchable relation $D = D_{\mathsf{test}}$. Query sources are disjoint from searched tuples and partitioned by source entity or time into training, validation, calibration, and test groups; benchmarks without source entities are partitioned directly before masking and configuration. Training groups form $\mathcal{Q}_{\mathsf{hist}}$. For threshold calibration, queries are organized into source groups $G_1, \ldots, G_{m_{\mathsf{cal}}}$. A group contributes either one query or the mean loss of its queries, and the calibration objective averages these group losses equally. Thus, m_{cal} reports the number of calibration groups, rather than the raw number of queries when groups contain multiple queries.

Values hidden by controlled masking are excluded from inference features and statistical support. Pre-mask development values provide the declared validation and calibration labels; hidden test values provide evaluation ground truth. The evaluator additionally uses complete values to construct QAE and value-dependent masks and to form diagnostic selectivity strata. These oracle operations do not expose hidden values to the methods or use test outcomes to select a design/threshold.

Missingness budgets. For each partition D_s, the incomplete-tuple budget is $B_s = \mathrm{round}(\rho|D_s|)$. When $m = 2$, the designated attributes split this budget approximately evenly over disjoint tuple sets; thus, ρ remains the overall incomplete-tuple rate and, at the default $w = 1$, every incomplete tuple has one masked cell. Mixed workloads designate one numerical and one categorical attribute; single-type workloads designate two attributes of that type when available. All designated attributes are fixed using only the schema and training workload.

MCAR samples each attribute's budget uniformly. HCE assigns a β_{exp} fraction by exposure-weighted sampling from embedding-only top-L candidates of training queries and fills the remainder uniformly; its exact construction is given below under *Historical candidate-exposure masks*. For QAE, let $G_s(q) = \mathsf{Ans}^*(q, D^{\star}_s)$ denote the complete-data filtered answer in partition s. The evaluator allocates an α_{ans} fraction of the same budget to tuples in these answers, masking a designated attribute referenced by the corresponding query, and fills any remainder uniformly. In the overall comparison, MCAR, HCE, and QAE share $\rho = 20\%$ and $w = 1$; HCE uses $\beta_{\mathsf{exp}} = 0.5$, and QAE uses $\alpha_{\mathsf{ans}} = 1$. All methods receive the same realized masks. QAE is an oracle-controlled stress mechanism, rather than an operational model of how natural missingness is generated.

Controlled repetitions. Within each repetition, all methods receive the same complete-data snapshot, query sources, controlled masks, and train–validation– calibration–test partitions. For MCAR, the missing-rate experiments use nested masks, so a tuple masked at a lower value of ρ remains masked at every larger value. For historical candidate exposure, all methods use the same training-query workload and the same seeded weighted-sampling draws at each β_{exp}. Unless explicitly varied, the query set, predicate distribution, k, missingness mechanism, and random seed are held fixed. External backends are configured using their validation-selected operating points and run with the same number of worker threads. For every reported exact-mode result, we additionally verify equality with brute-force top-k over the fixed predicted-eligible set $\widehat{E}_{\pi^\star,\theta}(q, D)$.

Historical candidate-exposure masks. Let $\mathcal{Q}_{\mathsf{hist}}(A)$ contain the training queries whose predicates reference a designated missing attribute A. For each partition D_s and query q, we rank only the tuples in D_s by embedding distance and let $C^{(s)}_L(q)$ denote its embedding-only top-L candidates. For tuple $t_i \in D_s$, we compute

$$s_s(i, A) = \sum_{q \in \mathcal{Q}_{\mathsf{hist}}(A)} \mathbf{1}[t_i \in C^{(s)}_L(q)].$$

An exposure-guided fraction β_{exp} of A's preallocated missingness budget is sampled without replacement with probability proportional to $s_s(i, A)$; if the positive-score pool is insufficient, the remainder is filled uniformly. The rest of the attribute budget is also sampled uniformly from unselected tuples. When $m = 2$, the two attribute-specific tuple sets are kept disjoint, so every incomplete tuple still has exactly one masked cell. This construction uses only the fixed training-query workload, observed embeddings, and predicate-attribute

identities; it never uses test queries, test answers, predicate outcomes, hidden scalar values, or method outputs.

Dataset-specific configurations. Table IX reports the deployed structure–policy configuration for each workload. Structure and policy choices are selected on validation data, after which the retained policy is refitted and the eligibility threshold $\theta^\star$ is selected on the disjoint calibration split.

The selected configurations are compact across workloads: all policies use $s_{\min} = 10$ without additional shrinkage, while dataset variation is absorbed mainly by the projection width, template composition, and calibrated eligibility threshold. The controlled Books-1M scale workload uses the same design procedure and selects a smaller leaf capacity than the corresponding Books workload.

B. Additional Effectiveness Results

Overall effectiveness. Table X reports the complete MCAR/HCE/QAE and natural-missingness comparison corresponding to the compact main-text Table II. Compared with lazy mean/mode imputation, MIFA improves average $F_1@10$ across datasets by 2.85, 3.00, and 9.24 percentage points under MCAR, HCE, and QAE, respectively.

Missingness rate and exposure. Figure 4 is reported in the main text. Here we summarize the corresponding setting and interpretation. Under MCAR, we vary $\rho \in \{10\%, 20\%, 30\%\}$ with $w = 1$. Under HCE, we fix (ρ, w) and vary $\beta_{\mathsf{exp}} \in \{0, 0.5, 1\}$ under the same incomplete-tuple budget, shifting missingness placement toward historically competitive candidates. The sweep results are averaged across three workloads; the mask construction is detailed in Appendix E-A.

Predicate workloads. We evaluate robustness to predicate complexity and result-set size by varying predicate type, conjunction arity $|\phi|$, and k under default HCE with exact execution. The arity sweep changes the number of predicate atoms in a conjunction; it is not the same as tuple-block missingness ($w > 1$), which jointly masks multiple attributes of a tuple. The main comparison uses each dataset's native single-attribute predicates; arity sweeps use datasets with sufficient applicable attributes and match selectivity where possible.

Alternative missingness patterns. At the default incomplete-tuple rate ρ, we compare the standard MCAR setting with attribute-skewed, tuple-block, and value-dependent missingness. All patterns use the same complete test snapshot and test queries in each repetition. We keep the total incomplete-tuple budget $B_s = \mathrm{round}(\rho|D_s|)$ fixed across patterns and report the resulting masked-cell rate separately, since tuple-block missingness may mask multiple cells in one incomplete tuple.

Attribute-skewed masks retain $w = 1$ but allocate the fixed total budget unequally across the designated attributes according to workload frequencies estimated from the training workload. Tuple-block masks set $w > 1$ and jointly hide multiple query-relevant attributes in each selected tuple. For value-dependent missingness, the evaluator uses the complete pre-mask value only to sample whether its cell is hidden; the hidden value is never provided to any evaluated method. The applicable primary baselines participate under the same masks. Table XII reports $F_1@10$ under the shared masks.

Table XII evaluates alternative missingness structures on mixed, numerical, and categorical workloads under the same overall incomplete-tuple budget. On WANDS, MIFA leads the strongest competing method by 3.2–4.9 $F_1@10$ points across all four mechanisms; on AppReviews, it remains within 0.3 points of the best result throughout. The harder case is tuple-block missingness on Amazon-multi, where MIFA drops from 0.906 under MCAR to 0.756 as multiple query-relevant attributes of the same tuple become simultaneously unobserved, whereas under value-dependent missingness it reaches 0.903 and exceeds the next best method by 2.5 points. Simultaneous missingness increases the residual target that a template must cover. Inference may then lack an applicable retained template or sufficient matching records; either outcome yields abstention. The F_1 results alone do not distinguish these causes. Thus, tuple-block missingness is an important limitation to examine alongside the default single-cell setting.

Oracle answer-targeted stress. We additionally vary requested answer-slot exposure in a separate oracle-controlled stress test. In QAE, α_{ans} controls the fraction of the global masking budget assigned to answer tuples; here, γ_{ans} controls the requested exposure within each query answer. These parameters define different masking procedures, so setting both to one does not make their realized masks identical. For each test query q, let

$$r_q = \mathrm{round}(\gamma_{\mathsf{ans}} |\mathsf{Ans}^*(q, D^*_{\mathsf{test}})|)$$

be its target number of exposed complete-data answer slots. Using a seeded query-wise round-robin procedure, the masker selects answer tuples whose query-relevant designated attributes remain available for masking, subject to the same global budget B_{test}, per-attribute budgets, and $w = 1$ constraint as the non-oracle settings. If both designated attributes are referenced by the query, the masker uses the attribute with the larger remaining budget, with seeded tie-breaking. Once the requested answer exposure is reached or no further eligible answer cell can be masked under these constraints, the remaining global budget is filled uniformly from unselected tuples.

Because one masked tuple can belong to the top-k answers of multiple queries, the realized exposure need not equal the target γ_{ans}. We therefore report

$$\eta_{\mathsf{ans}} = \frac{\#\{\text{Answer slots with a masked query-relevant cell}\}}{\#\{\text{complete-data answer slots}\}}.$$

We vary $\gamma_{\mathsf{ans}} \in \{0.25, 0.5, 0.75, 1\}$ and report η_{ans} and answer $F_1@10$. This exposure sweep is reported separately from the main QAE comparison and is not used for model selection.

Table XIII progressively targets complete-data top-k answer slots while keeping the overall incomplete-tuple budget fixed, with η_{ans} measuring the stress actually realized by each workload. On WANDS, MIFA leads the strongest competitor by 26.2–27.8 $F_1@10$ points throughout the sweep; on

Dataset	d_p	B_{leaf}	Ord.	Support context $X_s, [\mathsf{Loc}(A_{\mathbf{e}})]$	Target Y	m_{cal}	$\theta^\star$
WANDS	8	128	1	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{average_rating}	48	0.40
			2	{rating_count, subcategory}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{product_class}		
			3	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{product_class}		
Books	8	128	1	{rating_number, subcategory}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{average_rating}	10	0.81
			2	{subcategory}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{category}		
MovieLens	8	128	1	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{avg_rating}	50	0.41
			2	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{primary_genre}		
Yelp	8	128	1	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{stars}	50	0.59
			2	{city}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{state}		
H&M	16	128	1	{garment_group_name, section_name}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{department_name}	100	0.30
Amazon-multi	8	128	1	{store}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{main_category}	50	0.66
			2	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{store}		
AppReviews	8	128	1	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{review_length}	100	0.53
			2	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{star_rating}		
Amazon-C4	16	128	1	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{category}	200	0.33
ArXiv	8	128	1	{main_category, number_of_main_categories}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{number_of_sub_categories}	100	0.44
			2	{license, main_category}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{update_date}		
			3	$[\mathsf{Loc}(A_{\mathbf{e}})]$	{update_date}		
Books-1M	8	64	1	{rating_number, subcategory}, $[\mathsf{Loc}(A_{\mathbf{e}})]$	{average_rating}	10	0.53

TABLE IX: Dataset-specific MIFA structures, ordered retained support templates, and calibrated eligibility thresholds. All selected templates use $s_{\min} = 10$ and $\kappa_r = 0$, so these dataset-invariant settings are omitted here. The "Ord." column follows the template priority order fixed by the selected deployment policy.

AppReviews, realized exposure increases from 0.417 to 1.000 while MIFA decreases only from 0.945 to 0.920 and remains best at every operating point. The results are nonmonotonic on WANDS and Amazon-multi, even though the requested exposure increases. Requested exposure, realized exposure, and answer quality therefore need to be read separately. At $\gamma_{\mathsf{ans}} = 1$, QueryCls outperforms MIFA on Amazon-multi by 4.1 points, showing that MIFA's advantage is not uniform under this stress.

Predicate selectivity. For a test query q, we define its complete-data predicate selectivity as

$$\mathsf{sel}(q) = \frac{|E^*(q, D^*_{\mathsf{test}})|}{|D^*_{\mathsf{test}}|}.$$

For constructed predicates, constants are generated using training quantiles or frequencies; benchmark-provided filter constants are retained unchanged. After the queries have been fixed, the evaluator partitions them into the predeclared low-, medium-, and high-selectivity strata according to $\mathsf{sel}(q)$. The complete test values are used only for this diagnostic stratification and ground-truth computation; they are not exposed to any method or used to select an operating point.

All methods receive the same queries within each stratum. The applicable primary baselines use the same fixed masks and validation-selected configurations. Tables XIV and XV report answer quality, false-admission and false-rejection rates, and statistical resolutions per query. This diagnostic complements the controlled predicate-type and arity sweeps; selectivity strata can also differ in predicate composition.

Tables XIV and XV stratify the original main-workload queries by complete-data predicate selectivity and show that MIFA's relative gain changes across the three strata: MIFA leads throughout WANDS, leads in the low- and medium-selectivity strata of AppReviews, and gains mainly in the low-selectivity stratum of Amazon-multi. The execution workload also varies: Res./q decreases from 6.89 to 2.74–2.93 on AppReviews and from 7.12 to 0.04 on Amazon-multi. This count measures inference calls actually made during search, whereas FA and FR describe errors of the eligibility decisions. Under a fixed policy, exact execution preserves the predicted-eligible top-k regardless of how many evaluations it avoids. The observed quality differences concern the inference rule and query composition; they are not quality gains from geometric pruning or deferred evaluation.

Historical-evidence size. We evaluate MIFA using nested subsets containing 25%, 50%, 75%, and 100% of D_{tr}. For entity-based workloads, the subsets are formed by nested entity groups; for timestamped workloads, they are chronological prefixes. To study sensitivity to training-evidence size, the query workload, validation/calibration/test partitions, and controlled masks are held fixed across evidence sizes; in particular, $\mathcal{Q}_{\mathsf{hist}}$ is not resampled in this diagnostic.

At each evidence size, we repeat candidate discovery, joint design selection, policy refitting, structure construction, and

Dataset	Drop	Mean	MICE	MF	HC	R+B	QCls	UP	MIFA
(A) Predicate-level MCAR									
WANDS	0.803	0.826	0.825	0.849	0.837	0.855	0.863	0.108	**0.912**
Books	0.867	0.840	0.803	0.877	0.876	0.900	0.889	0.464	**0.930**
MovieLens	0.902	0.855	0.876	0.899	0.884	0.900	0.912	0.633	**0.920**
H&M	0.797	0.804	0.799	0.942	0.940	0.903	0.907	0.881	**0.973**
Yelp	0.880	0.836	0.874	0.938	0.925	0.955	0.912	0.518	**0.965**
Amazon-multi	0.838	0.570	0.851	0.899	0.881	0.838	0.882	0.214	**0.906**
AppReviews	0.793	0.942	0.935	0.935	0.793	0.935	0.958	0.919	**0.961**
Average	0.840	0.811	0.852	0.906	0.877	0.898	0.903	0.534	**0.938**
(B) Historical candidate exposure (HCE; $\beta_{\mathsf{exp}} = 0.5$)									
WANDS	0.786	0.818	0.813	0.833	0.788	0.773	0.851	0.104	**0.910**
Books	0.772	0.762	0.694	0.832	0.776	0.832	0.775	0.356	**0.895**
MovieLens	0.925	0.873	0.889	0.862	0.901	0.920	0.932	0.629	**0.934**
H&M	0.768	0.775	0.769	0.928	0.927	0.835	0.907	0.879	**0.970**
Yelp	0.827	0.791	0.823	0.951	0.892	0.936	0.890	0.460	**0.954**
Amazon-multi	0.850	0.533	0.850	0.891	0.885	0.847	0.889	0.199	**0.917**
AppReviews	0.718	0.934	0.927	0.929	0.724	0.877	0.951	0.916	**0.953**
Average	0.807	0.784	0.823	0.889	0.842	0.860	0.885	0.506	**0.933**
(C) Query-answer exposure (QAE; $\alpha_{\mathsf{ans}} = 1$)									
WANDS	0.000	0.153	0.153	0.287	0.416	0.568	0.546	0.101	**0.826**
Books	0.000	0.210	0.214	0.493	0.017	0.634	0.113	0.364	**0.791**
MovieLens	0.194	0.388	0.358	0.387	0.407	0.206	0.671	0.671	**0.811**
H&M	0.143	0.179	0.154	0.745	0.708	0.614	0.879	0.879	**0.960**
Yelp	0.001	0.222	0.134	0.799	0.266	0.444	0.544	0.367	**0.864**
Amazon-multi	0.000	0.228	0.017	0.333	0.038	0.025	0.463	0.235	**0.515**
AppReviews	0.000	0.788	0.765	0.760	0.000	0.738	0.913	0.906	**0.922**
Average	0.048	0.310	0.256	0.543	0.264	0.461	0.590	0.503	**0.813**
(D) Natural missingness									
OpenCatalog	0.933	0.944	0.939	0.939	0.939	0.944	0.972	0.972	**0.978**
Steam($w = 2$)	0.780	0.773	0.780	0.840	0.793	0.800	0.827	0.667	**0.853**
Funds($w = 2$)	0.769	0.783	0.749	0.757	0.769	0.783	0.769	0.737	**0.796**

TABLE X: Overall $F_1@10$ under three controlled missingness mechanisms ($\rho = 20\%$, $w = 1$, $k = 10$) and natural missingness. Mean/MF/HC/R+B/QCls denote MeanMode/MissForest/HoloClean/Raha+Baran/QueryCls; UP denotes UnknownPass; bold indicates the best result.

threshold calibration using only the corresponding training subset and the unchanged validation and calibration partitions. No tuple excluded from the selected training subset contributes support evidence. Because the offline procedure is repeated, this diagnostic includes changes in both support and the selected design. Table XVI reports support-index size, backoff statistics, predicate precision (P) and recall (R), false-admission (FA) and false-rejection (FR) rates, and

Setting	Drop	MeanMode	MICE	MissForest	HoloClean	Raha+Baran	QueryCls	MIFA
A. Predicate type — WANDS (F_1@10)								
Numerical	**0.926**	**0.926**	**0.926**	0.923	**0.926**	0.729	**0.926**	**0.926**
Categorical	0.886	0.886	0.862	0.872	0.873	0.874	0.886	**0.912**
Mixed	0.796	0.734	0.826	0.852	0.800	0.836	0.794	**0.909**
B. Conjunction arity — Books (F_1@10)								
$\|\phi\| = 1$	0.926	0.926	0.883	0.876	0.883	0.954	0.926	**0.984**
$\|\phi\| = 2$	0.856	0.850	0.784	0.873	0.854	0.910	0.856	**0.933**
$\|\phi\| = 3$	0.754	0.712	0.735	0.822	0.781	0.845	0.745	**0.883**
C. Requested result size — WANDS (F_1@k)								
$k = 5$	0.795	0.823	0.813	0.806	0.773	0.773	0.841	**0.898**
$k = 10$	0.786	0.818	0.813	0.833	0.787	0.773	0.851	**0.910**
$k = 20$	0.797	0.831	0.831	0.858	0.821	0.782	0.869	**0.931**
$k = 50$	0.816	0.852	0.852	0.880	0.853	0.809	0.890	**0.940**

TABLE XI: F_1 under varying predicate workloads with exact search and HCE ($\rho = 20\%$, $\beta_{\text{exp}} = 0.5$, $w = 1$). Bold and underlined values indicate best and second-best results.

Dataset	Method	MCAR	Attr.-skewed	Tuple-block	Value-dep.
WANDS	Drop	0.803	0.803	0.814	0.802
	MeanMode	0.826	0.837	0.814	0.830
	MICE	0.825	0.832	0.814	0.830
	MissForest	0.849	0.821	0.805	0.834
	HoloClean	0.837	0.833	0.814	0.854
	Raha+Baran	0.855	0.879	0.835	0.842
	QueryCls	0.863	0.882	0.830	0.858
	MIFA	**0.912**	**0.930**	**0.867**	**0.907**
AppReviews	Drop	0.793	0.793	0.806	0.802
	MeanMode	0.942	0.942	0.899	0.882
	MICE	0.935	0.934	0.891	0.943
	MissForest	0.935	0.936	0.882	0.941
	HoloClean	0.793	0.793	0.806	0.802
	Raha+Baran	0.935	0.924	0.874	0.954
	QueryCls	0.958	0.956	**0.929**	**0.964**
	MIFA	**0.961**	**0.959**	**0.929**	0.961
Amazon-multi	Drop	0.838	0.838	0.756	0.819
	MeanMode	0.570	0.565	0.431	0.522
	MICE	0.851	0.851	0.756	0.819
	MissForest	0.899	0.924	0.754	0.850
	HoloClean	0.881	0.914	0.756	0.848
	Raha+Baran	0.838	**0.933**	0.514	0.878
	QueryCls	0.882	0.888	**0.789**	0.857
	MIFA	**0.906**	0.870	0.756	**0.903**

TABLE XII: F_1@10 under alternative missingness mechanisms. Best and second-best displayed results within each dataset and mechanism are bolded and underlined, respectively; displayed ties share the same formatting.

F_1@10. This is a MIFA-only analysis because it measures the sensitivity of the proposed inference mechanism to the amount of historical support. We report representative results on WANDS and Amazon-multi, covering mixed categorical–numerical and categorical predicates.

Across the four evidence sizes, support coverage remains 1.0 and abstention remains zero on both workloads, while the retained template sets remain fixed at three templates on WANDS and two on Amazon-multi. WANDS remains stable across the sweep, with F_1@10 between 0.908 and 0.914 and a false-admission rate below 0.001 at every evidence size.

On Amazon-multi, F_1@10 reaches 0.917 with 50% historical evidence and remains unchanged through 100%, while using only 25% evidence reduces F_1@10 by about 0.010 to 0.907. All four evidence sizes require no hierarchical backoff, while the core shared support-index footprint grows approximately linearly from 0.31 to 1.24 MiB. In this sweep, the

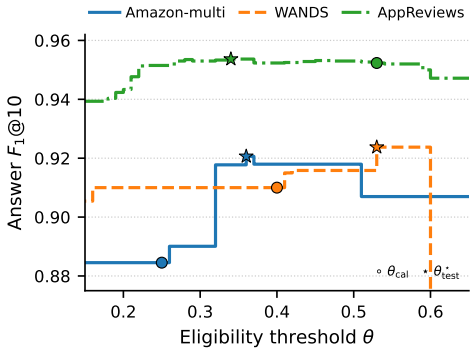


Fig. 7: Eligibility-threshold sensitivity on WANDS, AppReviews, and Amazon-multi. Circles mark θ_{cal} selected on D_{cal}; stars mark the diagnostic test-optimal $\theta^\star_{\text{test}}$.

displayed answer quality plateaus from 50% evidence onward, while the reported support-index footprint continues to grow.

Eligibility threshold. Keeping $\Gamma^\star$, the deployed structures $\mathcal{J}$, and $\pi^\star$ fixed, we evaluate every threshold in the predeclared set

$$\Theta = \{0, 0.01, \ldots, 0.99, 1\}.$$

We report the answer-level F_1@10 sweep and use predicate precision, recall, false-admission, and false-rejection rates to interpret threshold sensitivity.

Let $\theta_{\text{cal}} = \theta^\star$ denote the operating threshold selected on D_{cal}. For diagnosis only, we also report the test-optimal threshold $\theta^\star_{\text{test}}$, the threshold gap $|\Delta\theta|$, and the resulting answer-quality selection regret ΔF_1. The diagnostic evaluates both thresholds on the same test snapshot; F_1^{cal} denotes test F_1 at the calibration-selected threshold, not F_1 measured on the calibration snapshot. Only θ_{cal} is used for operating results; $\theta^\star_{\text{test}}$ is not used for model selection or deployment. This is a MIFA-only analysis. The reported test-snapshot regret is an empirical cross-snapshot diagnostic.

Figure 7 and Table XVII compare θ_{cal} with $\theta^\star_{\text{test}}$ while keeping $\Gamma^\star$, $\mathcal{J}$, and $\pi^\star$ fixed. The corresponding F_1@10 regrets are 0.001 on AppReviews, 0.014 on WANDS, and 0.000 on Amazon-multi. On WANDS, increasing θ from 0.40 to 0.53 raises predicate precision from 0.926 to 0.947 and reduces false admission from 0.00096 to 0.00066, while recall decreases from 0.980 to 0.968. On Amazon-multi, F_1@10 remains 0.917 across all 78 thresholds from $\theta = 0.23$ to 1.00, so $\theta_{\text{cal}} = 0.66$ and $\theta^\star_{\text{test}} = 1.00$ differ by 0.34 without answer-quality regret.

Thus, threshold sensitivity depends on the resulting predicate precision–recall balance: AppReviews is nearly unchanged around θ_{cal}, whereas Amazon-multi exhibits a broad answer-quality plateau. These results quantify empirical threshold sensitivity for the fixed design. The diagnostic does not provide a population-level calibration guarantee or establish that other methods received comparable tuning.

Dataset	γ_{ans}	η_{ans}	Drop	MeanMode	MICE	MissForest	HoloClean	Raha+Baran	QueryCls	MIFA
WANDS	0.25	0.873	0.130	0.233	0.233	0.400	0.520	0.148	0.560	**0.829**
	0.50	0.984	0.017	0.158	0.158	0.281	0.405	0.402	0.510	**0.789**
	0.75	0.998	0.002	0.145	0.145	0.276	0.421	0.357	0.560	**0.832**
	1.00	1.000	0.000	0.168	0.168	0.299	0.413	0.546	0.585	**0.847**
AppReviews	0.25	0.417	0.584	0.902	0.895	0.893	0.584	0.887	0.941	**0.945**
	0.50	0.674	0.326	0.859	0.844	0.837	0.326	0.845	0.929	**0.938**
	0.75	0.891	0.109	0.812	0.796	0.797	0.109	0.781	0.927	**0.928**
	1.00	1.000	0.000	0.780	0.748	0.738	0.000	0.736	0.917	**0.920**
Amazon-multi	0.25	0.990	0.010	0.233	0.034	0.321	0.046	0.401	0.501	**0.541**
	0.50	1.000	0.000	0.190	0.017	0.356	0.028	0.324	0.540	**0.601**
	0.75	1.000	0.000	0.179	0.024	0.396	0.005	0.353	0.425	**0.444**
	1.00	1.000	0.000	0.181	0.000	0.414	0.031	0.461	**0.561**	0.520

TABLE XIII: Oracle answer-targeted stress under fixed incomplete-tuple budgets. η_{ans} is the realized answer-slot exposure; remaining entries report F_1@10.

Dataset	**Sel.**	n	**Drop**	**MeanMode**	**MICE**	**MissForest**	**HoloClean**	**Raha+Baran**	**QueryCls**	MIFA
WANDS	Low	80	0.793	0.814	0.810	0.815	0.796	0.740	0.819	**0.859**
	Medium	80	0.793	0.799	0.788	0.845	0.821	0.741	0.893	**0.964**
	High	80	0.774	0.841	0.840	0.838	0.745	0.838	0.843	**0.908**
AppReviews	Low	200	0.760	0.857	0.850	0.864	0.763	0.826	0.891	**0.902**
	Medium	200	0.666	0.978	0.971	0.967	0.687	0.918	0.981	**0.982**
	High	200	0.729	0.967	0.960	0.957	0.723	0.889	**0.981**	0.976
Amazon-multi	Low	100	0.836	0.783	0.836	0.846	0.804	0.826	0.798	**0.856**
	Medium	100	0.829	0.568	0.829	0.905	0.907	0.829	**0.913**	0.909
	High	100	0.885	0.247	0.885	0.921	0.945	0.885	**0.957**	0.889

TABLE XIV: F_1@10 across predicate-selectivity strata; n is the number of queries in each stratum. Bold and underlined values indicate the best and second-best results in each row, respectively.

Dataset	**Sel.**	**FA**	**FR**	**Res./q**	F_1**@10**
WANDS	Low	0.00076	0.0746	21.36	0.859
	Medium	0.00076	0.0028	16.11	0.964
	High	0.00138	0.0201	30.80	0.908
AppReviews	Low	0.00670	0.0677	6.89	0.902
	Medium	0.03267	0.0123	2.74	0.982
	High	0.03884	0.0081	2.93	0.976
Amazon-multi	Low	0.00009	0.1148	7.12	0.856
	Medium	0.00069	0.0535	4.04	0.909
	High	0.00133	0.0366	0.04	0.889

TABLE XV: MIFA diagnostics on the same selectivity strata. FA and FR denote false-admission and false-rejection rates, and Res./q is the number of statistical resolutions per query.

Dataset	**Hist.**	$\lvert\mathcal{I}_{\text{mv}}\rvert$ **(MiB)**	**No backoff (%)**	**Depth**	**P**	**R**	**FA**	**FR**	F_1**@10**
WANDS	25%	0.52	92.9	2.92	.943	.971	0.00072	0.0293	0.914
	50%	1.04	86.6	4.92	.940	.967	0.00076	0.0331	0.908
	75%	1.55	86.7	5.09	.950	.966	0.00062	0.0339	0.913
	100%	2.07	87.6	5.00	.926	.980	0.00096	0.0204	0.910
Amazon-multi	25%	0.31	100.0	0.00	0.992	0.934	0.00007	0.0658	0.907
	50%	0.62	100.0	0.00	0.997	0.934	0.00003	0.0658	0.917
	75%	0.93	100.0	0.00	0.997	0.934	0.00003	0.0658	0.917
	100%	1.24	100.0	0.00	0.997	0.934	0.00003	0.0658	0.917

TABLE XVI: Historical-evidence sensitivity. P and R denote predicate precision and recall; FA and FR denote false-admission and false-rejection rates. No backoff is the share of statistical resolutions completed without hierarchical backoff, and Depth is the mean backoff depth when invoked.

C. Additional Efficiency and Scalability Results

Detailed end-to-end comparison. Table XVIII reports the full end-to-end comparison corresponding to the compact main-text Table III. In addition to answer quality and online latency/throughput, it reports index footprints, peak host memory (RAM), and index build time. We distinguish index build from the full offline preparation pipeline reported elsewhere in this appendix.

Runtime breakdown and predicate reuse. Table XIX profiles the mean online latency (ms/query) of the MIFA-CPU serving path, breaking it into fused execution stages. We report both steady-state serving (reusing populated support/inference state) and cohort-level cold-cache resets (including first-touch

Dataset	θ_{cal}	$\theta^{\star}_{\text{test}}$	$\lvert\Delta\theta\rvert$	F_1^{cal}	$F_1^{\text{test}\star}$	ΔF_1
WANDS	0.40	0.53	0.13	0.910	0.924	0.014
AppReviews	0.53	0.34	0.19	0.952	0.954	0.001
Amazon-multi	0.66	1.00	0.34	0.917	0.917	0.000

TABLE XVII: Calibration-selected and diagnostic test-optimal thresholds under the same fixed structure and policy. Here, $|\Delta\theta| = |\theta^{\star}_{\text{test}} - \theta_{\text{cal}}|$ and $\Delta F_1 = F_1@10(\theta^{\star}_{\text{test}}) - F_1@10(\theta_{\text{cal}})$.

construction). We further evaluate sensitivity to predicate reuse by issuing 153 distinct predicates on AppReviews under the same serving configuration.

Corpus-size scalability. Table XX reports per-method CPU and GPU results on nested 20K-, 100K-, and 1M-row prefixes of ArXiv-large at $d = 128$ under the same controlled HCE setting as the main scalability comparison. Baselines are served with HoloClean using their validation-selected operating points.

Efficiency under varying workloads. Using the Exp-1 runs, we evaluated MIFA's P95 CPU latency while varying missingness exposure β_{exp}, predicate type and arity, and k.

As shown in Figure 8, latency is more sensitive to predicate structure than to missingness placement or result-set size. Varying β_{exp} changes H&M latency only within 0.355–0.360 ms, and increasing k from 5 to 50 raises WANDS latency only from 1.376 to 1.421 ms. In contrast, latency rises from 0.165 to 1.536 ms from numerical to mixed predicates and from 0.124 to 0.635 ms as arity increases from one to three. Both k and missingness placement can change the competitive frontier and inference workload. Their measured latency effects are small in these sweeps, while the predicate-type and arity sweeps show larger variation.

All-dataset efficiency. We report MIFA on the nine standard HCE workloads and two ArXiv-large corpus scales using the CPU, one-GPU, and two-GPU serving paths. The one-GPU setting uses one serving process on one device; the two-GPU setting uses two concurrent serving replicas, with aggregate

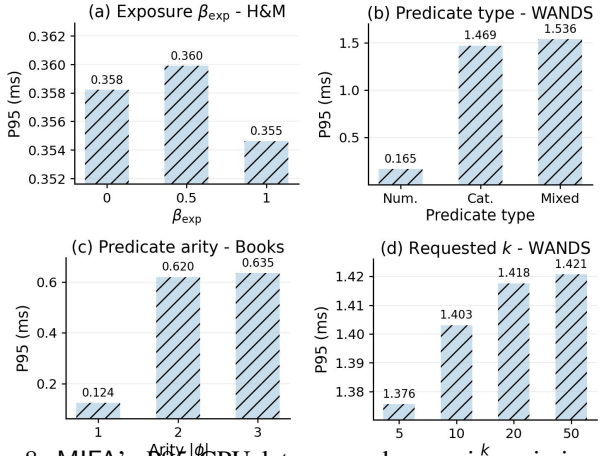


Fig. 8: MIFA's P95 CPU latency under varying missingness exposure, predicate type and arity, and result size k.

throughput and the slower-replica latency reported.

Table XXI reports a separate platform sweep over the nine standard workloads and two ArXiv-large scales, with workload-specific $F_1@10$ unchanged across execution platforms. On the nine standard workloads, the one-GPU path reaches 6.46K–57.42K QPS and two concurrent GPU replicas reach 12.82K–117.03K QPS, while P95 latency remains below 1.8 ms on all nine. The two-GPU throughput is close to twice the one-GPU throughput across these workloads, reflecting replica-level parallel serving rather than a change in query semantics or eligibility inference. These measurements demonstrate throughput scaling through independent replicas with unchanged displayed answer quality. They evaluate replication, rather than capacity scaling through partitioned multi-GPU search. Peak RSS measures host memory, not device-memory consumption.

Construction time and memory. We compare MIFA's offline construction with FAVOR and SIEVE paired with Drop, MeanMode, MICE, MissForest, HoloClean, and Raha+Baran on representative mixed, numerical, and categorical workloads. All methods use the same data snapshot, embeddings, scalar representation, hardware, and number of worker threads. Backend parameters are selected using the same validation search-quality targets as in the main experiments.

For MIFA, we separately report the time and persistent footprint of the plug-in ANN index $\mathcal{I}_e$, projected partition tree $\mathcal{T}_e$, shared support index $\mathcal{I}_{\text{mv}}$, and deployment policy $\pi^{\star}$. For $\mathcal{T}_e$, we separately attribute search metadata (*e.g.,* leaf lists and projected MBRs) and context-routing metadata where possible, while counting their shared topology only once. We additionally report policy discovery, validation-based joint selection, final refitting, and threshold-calibration time separately from serving-structure construction.

For each static baseline, we report wrapper fitting and materialization time separately from backend-index construction. For MeanMode, fitting includes estimating the training statistics, while materialization includes applying the selected values to the searchable snapshot. For MICE, MissForest, HoloClean, and Raha+Baran, we likewise report fitting or repair time and one-time materialization separately from the filtered-search backend. The end-to-end preparation time is the sum of these components.

Memory measurements report both peak resident memory during construction and the ready-to-query persistent footprint beyond the raw tuple and embedding arrays shared by all methods. For MIFA, we also report the fraction of persistent space consumed by each serving structure. This persistent-component breakdown measures the core serving structures; it does not by itself quantify warmed outcome caches, query-local workspace, or replicated device data. No offline construction or materialization time is included in the online query latency reported in Exp-2; its amortized contribution is reported separately under the fixed number of queries per construction or refresh.

Tables XXII–XXIV separate handler preparation, index

Method	Handler	F_1@10 ↑	Eng. Rec. ↑	P50 (ms) ↓	P95 (ms) ↓	QPS ↑	Index (MiB) ↓	RAM (GiB) ↓	Build (s) ↓
FAVOR	Drop	0.568	0.841	141.191	155.361	7.3	767.0	9.286	91.96
FAVOR	HC	0.568	0.841	113.059	127.984	8.9	767.0	9.334	95.69
SIEVE	Drop	0.662	1.000	30.655	41.121	36.1	21110.0	322.688	217.64
SIEVE	HC	0.662	1.000	30.537	41.600	35.7	33067.3	327.635	208.63
IRG	Drop	0.661	0.996	1.898	2.267	525.8	1800.5	8.886	671.74
IRG	HC	0.660	0.996	1.812	2.069	549.7	1800.4	8.980	747.57
UNG	Drop	0.675	0.958	3.529	204.283	16.1	3494.2	3.575	467.20
UNG	HC	0.696	0.959	3.692	205.710	16.0	3694.6	3.774	522.94
DSG	Drop	0.662	1.000	3.952	7.081	225.8	5739.4	21.642	4192.72
DSG	HC	0.662	1.000	6.450	7.959	155.1	5739.4	21.640	4160.97
WoW	Drop	0.661	0.999	6.473	8.853	150.4	2183.5	13.250	316.82
WoW	HC	0.661	0.999	6.982	9.447	138.3	2183.5	13.439	612.90
MIFA	Native	0.799	1.000	0.935	1.090	1108.6	257.7	8.908	50.57

TABLE XVIII: Full end-to-end comparison corresponding to the compact main-text Table III. Efficiency is measured on `ArXiv-large` at $d = 128$ under HCE ($\rho = 20\%$, $\beta_{\text{exp}} = 0.5$). IRG for iRangeGraph, HC for HoloClean.

	WANDS (240 Q)		H&M (600 Q)		AppReviews (600 Q)	
Component	Steady	Cold	Steady	Cold	Steady	Cold
Candidate / shortlist	0.139	0.145	–	–	1.256	1.277
Controller + final top-k	0.172	0.185	0.129	0.131	0.245	0.236
Exact-distance evaluation	0.860	0.876	0.292	0.308	2.994	2.997
Direct checks	0.104	0.136	0.346	0.355	0.882	0.902
Support / eligibility inference	0.005	4.467	0.007	0.045	0.006	1.049
Cache lookup	0.164	0.333	–	–	0.133	0.254
Total	**1.444**	**6.143**	**0.775**	**0.839**	**5.516**	**6.715**

TABLE XIX: Mean online latency (ms/query) of MIFA-CPU with cohort-level cold-cache resets; "–" denotes stages merged into fused kernels.

construction, and MIFA's offline design on mixed, numerical, and categorical workloads. For the static pipelines, backend construction remains a small part of end-to-end preparation once repair becomes substantial: for example, HoloClean contributes 31.79–58.56 s of repair time on AppReviews and WANDS, while Raha+Baran contributes 223.41–1401.51 s across the three workloads. MIFA spends 41.74, 37.47, and 174.50 s in total on WANDS, AppReviews, and H&M, respectively, and its phase breakdown shows that this cost is concentrated in validation-based joint selection and final refitting rather than in serving-structure construction, which takes 4.58–8.68 s. The persistent-state breakdown follows the structures selected for each workload: $\mathcal{I}_{\text{mv}}$ accounts for 90.79% and 71.17% of MIFA's state on WANDS and H&M, whereas $\mathcal{T}_e$ search metadata accounts for 80.17% on AppReviews; the deployment policy remains below 0.1 MiB in all three cases. Thus, MIFA places its main construction cost in offline structure–policy selection and refitting, while the deployed state consists primarily of the tree and shared support structures used by distance-ordered search and eligibility inference. With 10,000 queries per construction or refresh, the measured offline preparation corresponds to 4.17, 3.75, and 17.45 ms per query on the three workloads, respectively.

Additional scalability dimensions. In addition to varying $|D|$ in the main experiments, we vary the candidate-representation width and scalar-schema width $|\bar{\mathcal{A}}|$ in separate diagnostics.

Varying embedding dimension. We evaluate embedding dimension scaling with the Matryoshka representations of Qwen3-Embedding-0.6B on H&M (categorical), WANDS (mixed), and AppReviews (numerical). At each width, the truncated representation is used for candidate generation, and candidate distances are reranked using the same 1024-dimensional embeddings. Exact candidate reranking alone does not establish complete search coverage. For the fixed-policy exact executor in Theorem 1, changing only ANN seeds cannot change the returned tuple set; any candidate-limited execution requires its own quality measurement. Figure 9 reports a quality–resource trade-off across all three

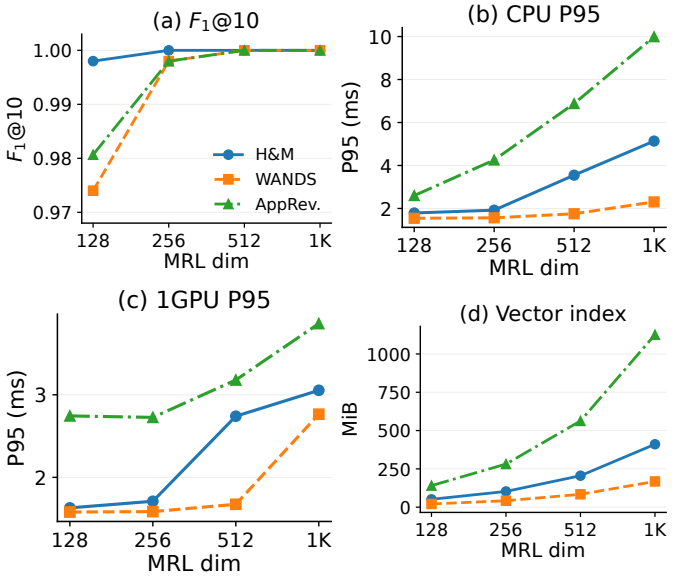


Fig. 9: Matryoshka-representation quality, retrieval cost, and vector-index footprint.

Method	20K				100K				1M			
	P50 (ms)	P95 (ms)	QPS	Index (MiB)	P50 (ms)	P95 (ms)	QPS	Index (MiB)	P50 (ms)	P95 (ms)	QPS	Index (MiB)
FAVOR	2.068	2.369	503.6	15.3	9.060	9.713	117.1	76.7	113.059	127.984	8.9	767.0
SIEVE	0.287	0.396	3722.7	427.0	2.322	3.116	472.1	2811.1	30.537	41.600	35.7	33067.3
iRangeGraph	0.574	0.661	1771.8	23.0	1.370	1.914	702.2	143.1	1.812	2.069	549.7	1800.4
UNG	0.223	2.775	1133.9	21.7	0.283	11.348	286.1	114.6	3.692	205.710	16.0	3694.6
DSG	0.451	0.561	2279.3	92.3	1.070	1.344	955.0	513.7	6.450	7.959	155.1	5739.4
WoW	0.429	0.503	2350.4	36.7	2.070	2.591	479.7	195.0	6.982	9.447	138.3	2183.5
MIFA-CPU	0.209	0.237	4821.1	59.4	0.282	0.343	3439.7	83.0	0.935	1.090	1108.6	257.7
MIFA-1GPU	1.334	1.434	6435.4	59.4	1.365	1.481	6129.3	83.0	1.191	1.618	3462.1	257.7
MIFA-2GPU	1.348	1.457	12245.9	59.4	1.406	1.594	11913.6	83.0	1.277	1.787	6106.4	257.7

TABLE XX: Full per-method/per-scale scalability results corresponding to the compact main-text Table V. Baselines are served with HoloClean.

Dataset	$\lvert D\rvert$	d	**Platform**	F_1**@10**	**P50 (ms)**	**P95 (ms)**	**QPS**	**Peak RSS (GiB)**
WANDS	6,349	1,024	CPU	0.910	1.371	1.443	727.4	2.04
			1-GPU	0.910	1.143	1.389	6,460.3	2.02
			2-GPU	0.910	1.172	1.395	12,816.3	4.04
Books	1,600	1,024	CPU	0.895	0.604	0.640	1,659.7	1.58
			1-GPU	0.895	1.084	1.273	7,285.7	1.46
			2-GPU	0.895	1.070	1.254	14,634.8	2.92
MovieLens	2,642	1,024	CPU	0.934	0.113	0.139	8,619.6	1.64
			1-GPU	0.934	0.665	0.799	9,863.4	1.51
			2-GPU	0.934	0.682	0.898	21,429.7	3.02
Yelp	11,890	1,024	CPU	0.954	1.265	2.125	884.0	2.26
			1-GPU	0.954	1.099	1.214	8,607.7	2.45
			2-GPU	0.954	1.079	1.232	17,370.7	4.91
H&M	21,025	1,024	CPU	0.970	0.157	0.422	5,213.8	3.32
			1-GPU	0.970	1.283	1.616	11,350.5	3.56
			2-GPU	0.970	1.265	1.598	22,811.3	7.12
Amazon-multi	5,359	1,024	CPU	0.917	1.169	1.209	855.8	1.89
			1-GPU	0.917	1.084	1.325	8,570.8	1.84
			2-GPU	0.917	1.093	1.184	17,113.2	3.67
AppReviews	57,613	1,024	CPU	0.953	0.806	1.259	1,231.5	7.14
			1-GPU	0.953	1.485	1.639	8,364.5	7.09
			2-GPU	0.953	1.472	1.615	16,707.1	14.23
Amazon-C4	12,000	1,024	CPU	0.768	0.420	1.754	1,307.1	2.30
			1-GPU	0.768	0.694	0.825	57,418.1	2.47
			2-GPU	0.768	0.720	0.858	117,030.3	4.92
ArXiv	20,000	4,096	CPU	0.742	1.201	2.060	776.1	9.06
			1-GPU	0.742	1.539	1.776	8,480.6	9.00
			2-GPU	0.742	1.541	1.794	17,036.0	18.01
ArXiv-large-1M	1,000,000	128	CPU	0.799	0.935	1.090	1,108.6	8.90
			1-GPU	0.799	1.272	1.605	3,781.0	8.70
			2-GPU	0.799	1.242	1.593	6,195.6	17.40
ArXiv-large-2.73M	2,735,264	128	CPU	0.817	1.717	2.011	583.2	11.57
			1-GPU	0.817	1.521	1.903	3,055.6	12.02
			2-GPU	0.817	1.531	1.958	5,926.9	24.03

TABLE XXI: MIFA efficiency on nine standard HCE workloads and two ArXiv-large scales using one or two RTX 4090 GPUs.

predicate regimes. At 512 dimensions, all three workloads match the displayed 1024-dimensional $F_1@10$ while halving the candidate-vector index footprint. At 256 dimensions, the equal-weight mean $F_1@10$ remains 0.9936, the candidate vector index is reduced by 75%, CPU P95 improves by a geometric-mean 2.20×, and 1GPU P95 by 1.59× relative to 1024 dimensions. The more aggressive 128-dimensional point retains mean $F_1@10 = 0.9851$, removes 87.5% of candidate-index bytes, and further improves CPU/1GPU P95 by 2.44×/1.66×. Shorter Matryoshka prefixes reduce the measured candidate-vector footprint and retrieval cost. The retained full-dimensional vectors are still needed for reranking, so the candidate-index reduction is not the total resident-memory reduction. The reported quality variation should be interpreted as a retrieval quality–cost diagnostic, separately from the fixed-policy exact execution guarantee.

Varying scalar-schema size. For the scalar-attribute experiment, we use nested attribute sets fixed before test execution. Predicate arity, incomplete-tuple rate, the default missingness mechanism, k, and the selectivity distribution are held fixed

Dataset	Backend	Handler	Fit/Repair	Materialize (s)	Build	E2E Prep.	Amort.@10K (ms/q)	Peak RSS (GiB)	Persistent (MiB)
WANDS	FAVOR	Drop	0.000	0.000	0.635	0.635	0.063	1.07	26.57
	SIEVE	Drop	0.000	0.000	0.312	0.312	0.031	1.70	192.10
	FAVOR	MeanMode	0.005	0.004	0.624	0.633	0.063	1.08	28.08
	SIEVE	MeanMode	0.005	0.004	0.314	0.323	0.032	1.70	198.65
	FAVOR	MICE	0.605	0.040	0.637	1.283	0.128	1.06	28.10
	SIEVE	MICE	0.605	0.040	0.392	1.038	0.104	1.91	267.04
	FAVOR	MissForest	6.598	0.710	0.629	7.938	0.794	1.88	28.10
	SIEVE	MissForest	6.598	0.710	0.361	7.670	0.767	1.88	226.39
	FAVOR	HoloClean	58.562	0.028	0.633	59.223	5.922	2.08	27.16
	SIEVE	HoloClean	58.562	0.028	0.318	58.907	5.891	2.08	210.09
	FAVOR	Raha+Baran	223.411	0.062	0.653	224.127	22.413	10.15	27.17
	SIEVE	Raha+Baran	223.411	0.062	0.339	223.813	22.381	10.15	214.09
	MIFA	Native	0.000	0.000	5.052	41.742	4.174	1.66	50.41
AppReviews	FAVOR	Drop	0.000	0.000	19.698	19.698	1.970	6.63	241.10
	SIEVE	Drop	0.000	0.000	3.334	3.334	0.333	6.63	474.47
	FAVOR	MeanMode	0.009	0.006	19.586	19.601	1.960	6.63	246.35
	SIEVE	MeanMode	0.009	0.006	3.505	3.520	0.352	6.63	494.55
	FAVOR	MICE	0.720	0.085	7.957	8.762	0.876	4.39	243.67
	SIEVE	MICE	0.720	0.085	5.428	6.233	0.623	11.96	2318.14
	FAVOR	MissForest	5.886	0.558	8.526	14.970	1.497	4.39	243.67
	SIEVE	MissForest	5.886	0.558	5.350	11.794	1.179	11.56	2228.52
	FAVOR	HoloClean	31.786	0.080	19.388	51.255	5.125	6.63	243.67
	SIEVE	HoloClean	31.786	0.080	3.388	35.254	3.525	6.63	488.22
	FAVOR	Raha+Baran	906.778	0.259	7.718	914.755	91.475	14.44	243.67
	SIEVE	Raha+Baran	906.778	0.259	2.490	909.527	90.953	14.44	490.89
	MIFA	Native	0.000	0.000	8.678	37.470	3.747	6.71	60.10
H&M	FAVOR	Drop	0.000	0.000	2.266	2.266	0.227	2.86	87.99
	SIEVE	Drop	0.000	0.000	0.332	0.332	0.033	2.86	94.11
	FAVOR	MeanMode	0.014	0.014	2.257	2.285	0.229	2.86	96.25
	SIEVE	MeanMode	0.014	0.014	0.343	0.371	0.037	2.86	107.84
	FAVOR	MICE	0.803	0.163	2.203	3.169	0.317	2.02	91.39
	SIEVE	MICE	0.803	0.163	0.371	1.337	0.134	2.02	97.65
	FAVOR	MissForest	4.093	0.248	2.271	6.612	0.661	2.03	91.38
	SIEVE	MissForest	4.093	0.248	0.393	4.734	0.473	2.02	98.78
	FAVOR	HoloClean	42.893	0.073	2.228	45.195	4.519	2.86	90.40
	SIEVE	HoloClean	42.893	0.073	0.332	43.299	4.330	2.86	96.53
	FAVOR	Raha+Baran	1401.510	0.184	2.255	1403.949	140.395	43.43	90.39
	SIEVE	Raha+Baran	1401.510	0.184	0.344	1402.039	140.204	43.43	99.46
	MIFA	Native	0.000	0.000	4.578	174.504	17.450	3.12	53.02

TABLE XXII: Offline construction and memory accounting on representative mixed, numerical, and categorical workloads. For MIFA, Build reports serving-structure construction only, whereas E2E Prep. equals the full offline total in Table XXIV. Amort.@10K uses 10,000 queries per construction or refresh.

as closely as possible, so that the experiment measures the effect of a larger scalar schema rather than a different query workload. The designated missing-attribute set remains fixed whenever it is contained in the current scalar schema. Each method repeats its validation-only configuration selection at every scale; test results are not used to reuse or modify an operating point.

Every applicable static filtered-ANN backend participates with Drop and HoloClean. We report construction time, peak and persistent memory, median and P95 latency, throughput, exact-distance counts, and $F_1@10$. The quality metric ensures that lower construction or query cost is not obtained merely by using a lower-quality operating point. This scalar-schema-only ablation uses the controlled MovieLens and Yelp relations, containing $2{,}642$ and $11{,}890$ searchable tuples, respectively. We use P50, P95, and QPS here to measure changes across $|\bar{\mathcal{A}}|$ within these controlled workloads, while the main tables provide the cross-method end-to-end efficiency comparison.

Tables XXV and XXVI vary $|\bar{\mathcal{A}}|$ from two to four while fixing the mixed predicate workload on MovieLens and Yelp.

Dataset	Serving component	Construction (s)	Persistent (MiB)	Share (%)
WANDS	$\mathcal{I}_e$	0.000	0.000	0.00
	$\mathcal{T}_e$ search	0.719	4.511	8.95
	$\mathcal{T}_e$ context	0.058	0.121	0.24
	$\mathcal{I}_{\mathsf{mv}}$	0.023	45.773	90.79
	$\pi^\star$	4.247	0.009	0.02
AppReviews	$\mathcal{I}_e$	0.000	0.000	0.00
	$\mathcal{T}_e$ search	7.473	48.179	80.17
	$\mathcal{T}_e$ context	0.619	1.099	1.83
	$\mathcal{I}_{\mathsf{mv}}$	0.067	10.757	17.90
	$\pi^\star$	0.486	0.064	0.11
H&M	$\mathcal{I}_e$	0.000	0.000	0.00
	$\mathcal{T}_e$ search	3.585	14.884	28.07
	$\mathcal{T}_e$ context	0.271	0.401	0.76
	$\mathcal{I}_{\mathsf{mv}}$	0.087	37.737	71.17
	$\pi^\star$	0.622	0.002	0.00

TABLE XXIII: Construction time and persistent-space attribution of the MIFA serving structures. The shared tree topology is counted once in the search row. For the configuration profiled here, support memory includes prebuilt body masks, and no separate ANN graph is stored.

Dataset	Discovery	Joint Sel.	Refit	Calibration (s)	Serving Build	Total
WANDS	0.807	17.691	17.240	0.953	5.052	41.742
AppReviews	0.253	10.091	9.326	9.123	8.678	37.470
H&M	1.334	75.773	75.004	17.815	4.578	174.504

TABLE XXIV: Offline phase breakdown of MIFA on the same three workloads.

Dataset	$\lvert\bar{\mathcal{A}}\rvert$	Method	Handler	F_1@10	P50 (ms)	P95 (ms)	QPS	Dist./q
MovieLens	2	MIFA	Native	**0.874**	0.745	0.796	1337.1	2642.0
	2	FAVOR	Drop	0.849	0.493	0.754	1769.0	387.3
	2	FAVOR	HoloClean	0.847	0.504	0.780	1724.6	388.6
	2	SIEVE	Drop	0.850	0.023	0.041	37802.9	42.0
	2	SIEVE	HoloClean	0.849	0.024	0.042	36018.1	42.6
	3	MIFA	Native	**0.874**	0.724	0.769	1374.9	2642.0
	3	FAVOR	Drop	0.849	0.498	0.774	1741.7	387.3
	3	FAVOR	HoloClean	0.847	0.505	0.772	1716.9	388.5
	3	SIEVE	Drop	0.850	0.024	0.044	36174.2	42.0
	3	SIEVE	HoloClean	0.849	0.023	0.043	36978.9	42.5
	4	MIFA	Native	**0.874**	0.725	0.778	1371.7	2642.0
	4	FAVOR	Drop	0.849	0.501	0.774	1741.4	387.3
	4	FAVOR	HoloClean	0.847	0.497	0.780	1734.2	388.6
	4	SIEVE	Drop	0.850	0.024	0.044	35869.4	42.0
	4	SIEVE	HoloClean	0.849	0.023	0.043	36825.6	42.6
Yelp	2	MIFA	Native	**0.786**	2.123	2.192	470.3	11890.0
	2	FAVOR	Drop	0.716	1.576	1.811	625.6	525.7
	2	FAVOR	HoloClean	0.716	1.596	1.862	613.7	525.7
	2	SIEVE	Drop	0.765	0.047	0.095	17962.1	163.3
	2	SIEVE	HoloClean	0.765	0.046	0.093	18557.2	163.3
	3	MIFA	Native	**0.786**	2.097	2.160	475.8	11890.0
	3	FAVOR	Drop	0.716	1.614	1.872	615.1	525.7
	3	FAVOR	HoloClean	0.716	1.617	1.892	604.8	525.7
	3	SIEVE	Drop	0.765	0.047	0.091	18274.7	163.3
	3	SIEVE	HoloClean	0.765	0.046	0.091	18576.3	163.3
	4	MIFA	Native	**0.786**	2.131	2.193	468.6	11890.0
	4	FAVOR	Drop	0.716	1.623	1.897	608.7	525.7
	4	FAVOR	HoloClean	0.716	1.603	1.871	611.3	525.7
	4	SIEVE	Drop	0.765	0.047	0.097	17872.5	163.3
	4	SIEVE	HoloClean	0.765	0.045	0.093	18359.9	163.3

TABLE XXV: Scalar-schema-width quality and online execution on MovieLens and Yelp.

Across all methods, F_1@10 is unchanged across the three widths; within each method, the width-2 to width-4 P95 change is at most $0.020\,\mathrm{ms}$ on MovieLens and $0.085\,\mathrm{ms}$ on Yelp, and the exact-distance count remains unchanged. For

Dataset	$\lvert\bar{\mathcal{A}}\rvert$	Method	Handler	Build (s)	Peak RSS (GiB)	Persistent (MiB)	Index (MiB)
MovieLens	2	MIFA	Native	1.022	1.53	12.37	12.37
	2	FAVOR	Drop	0.234	0.81	846.08	11.06
	2	FAVOR	HoloClean	0.242	0.81	845.96	11.06
	2	SIEVE	Drop	0.119	1.05	886.67	73.17
	2	SIEVE	HoloClean	0.121	1.05	886.50	72.81
	3	MIFA	Native	1.023	1.61	12.48	12.48
	3	FAVOR	Drop	0.241	0.81	846.11	11.06
	3	FAVOR	HoloClean	0.235	0.81	846.05	11.06
	3	SIEVE	Drop	0.143	1.05	890.71	77.22
	3	SIEVE	HoloClean	0.121	1.05	891.15	77.14
	4	MIFA	Native	1.014	1.69	12.64	12.64
	4	FAVOR	Drop	0.235	0.81	846.23	11.06
	4	FAVOR	HoloClean	0.239	0.81	846.37	11.06
	4	SIEVE	Drop	0.127	1.06	897.94	83.99
	4	SIEVE	HoloClean	0.106	1.06	897.31	83.61
Yelp	2	MIFA	Native	1.611	2.25	55.15	55.15
	2	FAVOR	Drop	1.124	1.40	1386.03	49.76
	2	FAVOR	HoloClean	1.183	1.40	1386.98	49.76
	2	SIEVE	Drop	0.220	1.41	1292.39	54.36
	2	SIEVE	HoloClean	0.244	1.41	1295.68	54.37
	3	MIFA	Native	1.642	2.58	55.63	55.63
	3	FAVOR	Drop	1.128	1.40	1387.69	49.76
	3	FAVOR	HoloClean	1.169	1.40	1387.84	49.76
	3	SIEVE	Drop	0.225	1.41	1298.26	54.49
	3	SIEVE	HoloClean	0.281	1.41	1297.59	55.36
	4	MIFA	Native	1.627	2.26	56.34	56.34
	4	FAVOR	Drop	1.138	1.40	1389.54	49.76
	4	FAVOR	HoloClean	1.161	1.40	1388.80	49.76
	4	SIEVE	Drop	0.268	1.51	1372.24	128.05
	4	SIEVE	HoloClean	0.277	1.51	1376.12	128.72

TABLE XXVI: Scalar-schema-width construction and memory on MovieLens and Yelp. Persistent and Index denote the reported persistent and index footprints, respectively; Peak RSS is process memory.

MIFA, the selected structure, policy, and threshold also remain unchanged, with 25.92 statistical resolutions per query on MovieLens and 52.11 on Yelp at all three widths. The stable execution work follows from the fixed predicate and the unchanged serving configuration, while the additional scalar attributes are reflected in the maintained index state: MIFA's support-index component grows from 0.34 to 0.61 MiB on MovieLens and from 1.53 to 2.72 MiB on Yelp, with the candidate-provider and tree components unchanged; correspondingly, its reported auxiliary persistent state grows from 12.37 to 12.64 MiB and from 55.15 to 56.34 MiB. The static indexes exhibit backend-specific state growth, most visibly for SIEVE, whose index grows from 73.17 to 83.99 MiB on MovieLens and from 54.36 to 128.05 MiB on Yelp. Thus, under a fixed mixed-predicate workload, increasing scalar-schema width primarily changes the amount of maintained scalar state rather than the geometric or eligibility work performed per query, matching MIFA's design in which additional scalar evidence is incorporated through the shared support index while the selected distance-ordered execution path remains fixed.

D. Additional Ablation Details

Serving-structure ablation. Table VI is reported in the main text. Here we summarize the controlled variants and how to interpret the results. We ablate $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\mathsf{mv}})$ under the same fused CPU backend with the hard-pass direct-top-k shortcut disabled. Only MIFA$_{-\mathcal{I}_e}$ (no ANN seeding) and MIFA$_{-\mathcal{I}_{\mathsf{mv}}}$ (support-index scanning of D_{tr}) preserve MIFA's eligibility policy; MIFA$_{-\mathcal{T}_e}$ and MIFA$_{-\mathcal{J}}$ additionally replace location-aware inference with ScalarOnly, changing the inference policy.

Hierarchical inference. Figure 6(a–b) reports the context-quality and context-selection behavior of hierarchical backoff; LeafOnly abstains when leaf support is insufficient, whereas MIFA backs off to broader contexts.

Inference–search scheduling. Figure 6(c–d) reports execution-order ablations under a fixed eligibility rule, comparing MIFA with InferenceFirst and SearchFirst in both latency and inference work.

Eligibility threshold. Figure 6(f) reports the empirical calibration-stability sweep under a fixed inference policy; detailed threshold sensitivity is in Section E-B.

E. Approximate Execution and Maintenance

Approximate execution. We compare exact mode ($\epsilon = 0$) with predeclared positive ϵ values, reporting F_1, P95 latency (ms), expanded nodes, exact-distance work (Dist./N), eligibility-resolution requests, exact-mode result-set recall (R-Rec.), and the maximum boundary ratio (Max-d_k), defined as the maximum ratio of the returned k-th distance to its exact-mode counterpart, used to verify the $(1+\epsilon)$ distance bound.

Data	ϵ	F_1	P95 (ms)	Node/q	Dist./N	Req./q	R-Rec.	Max-d_k
H&M	0	0.979	75.79	1882.5	0.813	38.27	1.000	1.0000
	0.005	0.975	22.19	246.8	0.072	39.82	0.995	1.0046
ArXiv	0	0.807	155.60	2047.0	0.998	16.03	1.000	1.0000
	0.0025	0.806	91.46	1750.0	0.774	18.36	0.988	1.0024

TABLE XXVII: Approximate execution on H&M and ArXiv. Req./q counts eligibility-resolution requests, including cache hits.

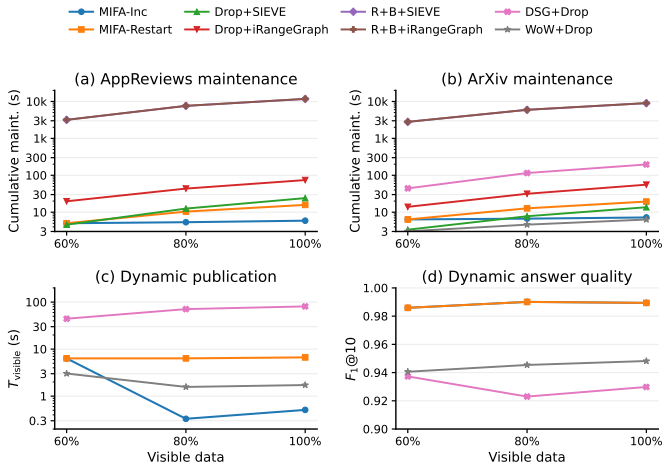


Fig. 10: Incremental-ingestion performance as the visible fraction of each stream increases from 60% to 100%.

Incremental maintenance. We evaluate incremental ingestion by building an initial version from the earliest 60% of timestamp-ordered tuples and appending batches to reach 80% and 100% visibility. MIFA-Incremental keeps the base immutable and updates only the ANN delta state and $\Delta\mathcal{T}_e$, whereas MIFA-Restart rebuilds all searchable structures. Both reuse $\pi^\star$ and $\theta^\star$, while MIFA-Restart reconstructs inference contexts and support structures and MIFA-Incremental retains those of the initial base; appended tuples are immediately searchable but contribute statistical support only after refresh. We compare with restart pipelines $\{\mathsf{Drop}, \mathsf{Raha{+}Baran}\} \times \{\mathsf{SIEVE}, \mathsf{iRangeGraph}\}$ and, for numerical-range streams, DSG and WoW paired with Drop.

Method	F_1@10 ↑	P95 (ms) ↓	T_{visible} (s) ↓	Cum. maint. (s) ↓
D+SIEVE	0.716	8.39	11.53	24.06
D+iRG	0.699	1.17	30.58	74.16
RB+SIEVE	0.549	10.02	4132.36	11723.60
RB+iRG	0.539	1.01	4144.40	11757.59
DSG+D	0.470	**0.14**	91.80	209.06
WoW+D	0.553	0.22	4.41	17.84
MIFA-R	0.756	1.03	5.41	15.78
MIFA-I	**0.763**	1.76	**0.53**	**5.88**

TABLE XXVIII: Incremental-serving on AppReviews. MIFA-R and MIFA-I denote restart and incremental maintenance. In separate tests with contexts and support fixed, incremental maintenance and rebuilding return identical ordered result IDs in all 2,880 comparisons.